



FACULTAD DE INGENIERIA

Universidad de Buenos Aires

Data Analytics para la estimación de la presión de yacimientos de petróleo

Director: Dr. Hernán Merlino (hmerlino@fi.uba.ar)

Co-directora: Dra. Gabriela Savioli (gsavioli@fi.uba.ar)

Integrantes

Nombre	Email	Padrón
Alan Valdevenito	avaldevenito@fi.uba.ar	107585
Franco Bragantini	fbragantini@fi.uba.ar	97190
Mateo Rico	mrico@fi.uba.ar	108127
Máximo Palopoli	mpalopoli@fi.uba.ar	108755

Índice

1. Aval del Director	1
2. Resumen	2
3. Palabras clave	3
4. Abstract	4
5. Keywords	5
6. Agradecimientos	6
7. Introducción	7
8. Estado del Arte	8
9. Problema detectado y/o faltante	10
10. Solución implementada	11
10.1. Arquitectura	11
10.1.1. Aplicación Web	11
10.1.2. Modelo de Machine Learning	12
11. Metodología aplicada	19
11.1. Desarrollo incremental y entregables	19
11.2. Control de versiones	19
11.3. Gestión del proceso y seguimiento	19
11.4. Automatización	20
11.5. Criterios de aceptación	20
11.6. Artefactos de gestión y comunicación	20
11.7. Indicadores de calidad	20
12. Herramientas externas utilizadas	21
13. Experimentación y/o validación	23
13.1. Evaluación y análisis comparativo de los modelos de machine learning	23
13.1.1. Modelo baseline	23
13.1.2. Random Forest	26

13.1.3. XGBoost	28
13.1.4. LSTM	31
13.1.5. Comparación final entre modelos	33
13.2. Validación de generalización multi-campo (leave-one-field-out)	35
14. Cronograma de las actividades realizadas	37
14.1. Cronograma del segundo cuatrimestre 2025	38
14.2. Cronograma del primer cuatrimestre 2026	39
15. Riesgos materializados y Lecciones aprendidas	41
15.1. Riesgos materializados	41
15.1.1. Dependencia de datos de la industria	41
15.2. Lecciones aprendidas	41
15.2.1. Comprensión del dominio	41
15.2.2. Flexibilidad y adaptabilidad	41
15.2.3. Importancia de la validación continua	42
16. Impactos sociales y ambientales del proyecto	43
16.1. Impacto económico	43
16.2. Impacto social	43
16.3. Impacto ambiental	43
16.4. Consideración sobre el alcance	43
17. Trabajos futuros	44
17.1. Física embebida (PINN de balance de materiales)	44
17.2. Calibración few-shot	44
17.3. Más campos y datos reales	44
17.4. Validación contra métodos físicos (PTA/EBM)	45
17.5. Mejoras en la aplicación web	45
18. Conclusiones	46
19. Aportes individuales	48
20. Anexos	49
21. Referencias	50

1 Aval del Director

En Buenos Aires a [día] de [mes] del año [año] se reúnen Dr. Hernán Merlino con los estudiantes de la carrera de Ingeniería en Informática: Franco Julián Bragantini, Máximo Palopoli, Mateo Julián Rico y Alan Valdevenito. El objetivo de la reunión es revisar el Informe Final del Trabajo Profesional de Ingeniería en Informática.

Luego de haber leído el informe del Trabajo Profesional **Data Analytics para la estimación de la presión de yacimientos de petróleo**, el Director del mismo, Dr. Hernán Merlino, declara que el mismo se corresponde con el trabajo realizado a lo largo del proyecto y avala el documento presentado como Informe Final, tanto en su completitud como en la claridad de redacción.

Firma y aclaración del director: [firma y aclaración]

Firmas y aclaraciones de los estudiantes: [firmas y aclaraciones]

2 Resumen

La estimación precisa de la presión en yacimientos de petróleo es un factor crítico para la gestión eficiente de la producción de hidrocarburos. Los métodos tradicionales, basados en la física de reservorios, presentan limitaciones frente al creciente volumen y complejidad de los datos disponibles. En este contexto, surge la necesidad de explorar nuevas herramientas que permitan realizar predicciones rápidas, robustas y escalables.

Este proyecto consiste en el desarrollo de un sistema para estimar la presión de yacimientos de petróleo, basado en técnicas de *Data Analytics* y *Machine Learning*. La solución integra un modelo predictivo que consume y procesa datos físicos del reservorio, junto con una interfaz web que facilita su uso por parte de ingenieros sin formación específica en ciencia de datos.

3 Palabras clave

Yacimiento de petróleo, machine learning, estimación de presión, física de reservorio, recuperación secundaria.

4 Abstract

Accurate estimation of reservoir pressure is a key factor in the efficient management of hydrocarbon production. Traditional methods, based on reservoir physics, present limitations when dealing with the increasing volume and complexity of modern data. In this context, new approaches are required to provide fast, robust, and scalable predictions.

This project consists of the development of a system to estimate the pressure of oil reservoirs, based on *Data Analytics* and *Machine Learning* techniques. The solution integrates a predictive model that processes reservoir data, along with a web interface designed for engineers without data science expertise.

5 Keywords

Oil reservoir, machine learning, pressure estimation, reservoir physics, secondary recovery.

6 Agradecimientos

Texto de agradecimientos (sección opcional).

7 Introducción

La extracción de petróleo es un proceso que se lleva a cabo en formaciones subterráneas conocidas como yacimientos, donde el hidrocarburo, junto con otras sustancias como gas y agua, se encuentra almacenado en rocas porosas bajo condiciones de presión natural.

La gestión eficaz y la explotación económica de dichos yacimientos dependen fundamentalmente de una comprensión precisa de sus propiedades y comportamientos. Entre todos los parámetros que gobiernan la producción de hidrocarburos, la **presión** de yacimiento se erige como la variable más crítica.

Durante la fase inicial de la producción (recuperación primaria), la presión del reservorio es la fuerza motriz que empuja los fluidos desde la roca hasta la superficie [1]. Sin embargo, a medida que avanza la explotación del yacimiento, la presión disminuye progresivamente, por lo que para sostener la producción es necesario mantener o incrementar la presión mediante diferentes técnicas, entre las que destacan los métodos de **recuperación secundaria**, como la inyección de agua o gas.

En este contexto, la correcta estimación de la presión es la piedra angular de la ingeniería de yacimientos moderna, influyendo en cada decisión, desde la perforación del primer pozo exploratorio hasta el abandono final del campo.

En este documento presentamos un MVP de un sistema para estimar la presión de reservorios mediante un modelo predictivo de machine learning.

En las siguientes secciones se describe en mayor detalle el problema a resolver, la solución implementada, la metodología utilizada, el impacto del proyecto en la industria, la planificación del desarrollo y los mecanismos de validación del sistema.

8 Estado del Arte

Antes de la llegada del análisis de datos a gran escala, la ingeniería de yacimientos se basaba en dos pilares metodológicos: el Análisis de Transientes de Presión (PTA por sus siglas en inglés) [2] y la Ecuación de Balance de Materiales (EBM) [3].

El PTA es un enfoque dinámico que analiza la respuesta de la presión ante perturbaciones en el pozo. Incluye pruebas como drawdown y buildup, que permiten calcular parámetros locales como la permeabilidad promedio, el factor de daño (skin) y la presión promedio del área de drenaje. Su fortaleza radica en la alta resolución cerca del pozo, aunque con alcance espacial limitado.

La EBM, por su parte, aplica la conservación de la masa tratando al yacimiento como un “tanque” a volumen constante. Facilita la estimación de POES/GOES, la identificación del mecanismo de empuje y la predicción del comportamiento futuro. Su análisis suele apoyarse en métodos gráficos como P/Z vs. Gp, aunque asume homogeneidad en el yacimiento.

Ambos enfoques físicos, basados en los principios fundamentales de la física de fluidos en medios porosos, son complementarios y representan las herramientas clásicas que han permitido la caracterización y gestión de yacimientos durante décadas.

En la actualidad, la industria vive una transición hacia el paradigma basado en datos [4], impulsada por el crecimiento del volumen de datos proveniente de fuentes como adquisición sísmica, registros de perforación, sensores permanentes de pozos y laboratorios.

En este contexto, los métodos de data analytics y machine learning se posicionan como herramientas capaces de aprender directamente de los datos históricos. Estos modelos permiten:

- Optimizar la producción, identificando patrones ocultos y anticipando problemas como la irrupción de agua o arena.
- Mejorar la caracterización de yacimientos, integrando múltiples fuentes para generar modelos más precisos (ejemplo: incrementos del 30 % en la precisión de modelos geológicos reportados en la industria).
- Habilitar el mantenimiento predictivo, anticipando fallos en equipos críticos mediante análisis de datos de sensores.
- Reducir costos y riesgos, al mejorar la toma de decisiones en tiempo real y aumentar la seguridad operacional.

Entre los algoritmos más destacados se encuentran:

- **Redes Neuronales Artificiales (ANN):** sobresalientes en mapeos complejos, con resultados que superan correlaciones empíricas.

- **Random Forest y XGBoost:** robustos y eficientes para predicción de presiones críticas (p. ej., presión de burbuja, presión de poro).
- **Support Vector Regression (SVR):** eficaz en espacios de alta dimensionalidad, mejorando su precisión mediante optimización de hiperparámetros.
- **Redes LSTM:** especialistas en series temporales, ideales para pronósticos de producción y presión.

El avance más reciente es la integración de datos y física en enfoques híbridos. Los proxies de ML permiten generar predicciones en milisegundos tras ser entrenados con simulaciones numéricas, y las Redes Neuronales Informadas por la Física (PINNs) [5] incorporan ecuaciones diferenciales en su entrenamiento, garantizando consistencia física aun en ausencia de datos.

9 Problema detectado y/o faltante

Los métodos convencionales, aunque robustos y basados en principios físicos sólidos, muestran limitaciones significativas para manejar la escala y complejidad de los datos modernos:

- **Costo computacional:** La simulación numérica de yacimientos, el estándar de oro para el modelado predictivo, es un proceso extremadamente intensivo en términos computacionales. Una sola simulación de un modelo de alta resolución puede tardar horas o incluso días en completarse, lo que hace inviable la exploración exhaustiva de incertidumbres o la optimización en tiempo real [6].
- **Simplificación excesiva:** Los modelos analíticos, como la EBM, deben hacer suposiciones simplificadoras (por ejemplo, tratar el yacimiento como un tanque homogéneo) para ser matemáticamente tratables. Esto puede pasar por alto la heterogeneidad espacial compleja que a menudo controla el flujo de fluidos en la realidad [7].
- **Análisis manual y lento:** La interpretación de pruebas de pozo o la calibración de un simulador (ajuste histórico) son procesos que tradicionalmente han requerido un esfuerzo manual significativo por parte de ingenieros expertos, lo que los convierte en procesos lentos y potencialmente subjetivos [8].

En consecuencia, existe una brecha entre las necesidades actuales (predicciones rápidas, robustas y capaces de integrar múltiples fuentes de datos) y las capacidades de las metodologías tradicionales.

Esta brecha justifica la **exploración de nuevos enfoques** basados en Machine Learning, que permitan aprovechar datos históricos para generar estimaciones de presión más ágiles, transparentes y escalables.

10 Solución implementada

La solución implementada es una aplicación web que permite realizar estimaciones de la presión de un reservorio mediante un modelo de machine learning. El sistema toma como entrada un conjunto de datos cargados por el usuario y, a partir de ellos, genera una predicción de la presión del reservorio.

La información requerida comprende datos de producción, que corresponden a series temporales registradas a lo largo de la vida del reservorio, datos físicos del reservorio, e información termodinámica del petróleo del reservorio.

Una vez cargados los datos, el modelo procesa estas distintas fuentes de información para producir la estimación de presión. La aplicación presenta los resultados de forma visual, mediante gráficos que muestran tanto la presión estimada como la incertidumbre asociada a dicha estimación, permitiendo al usuario interpretar no solo el valor predicho sino también el grado de confianza del modelo.

10.1 Arquitectura

El sistema está compuesto una aplicación web (frontend y backend) y un modelo de machine learning.

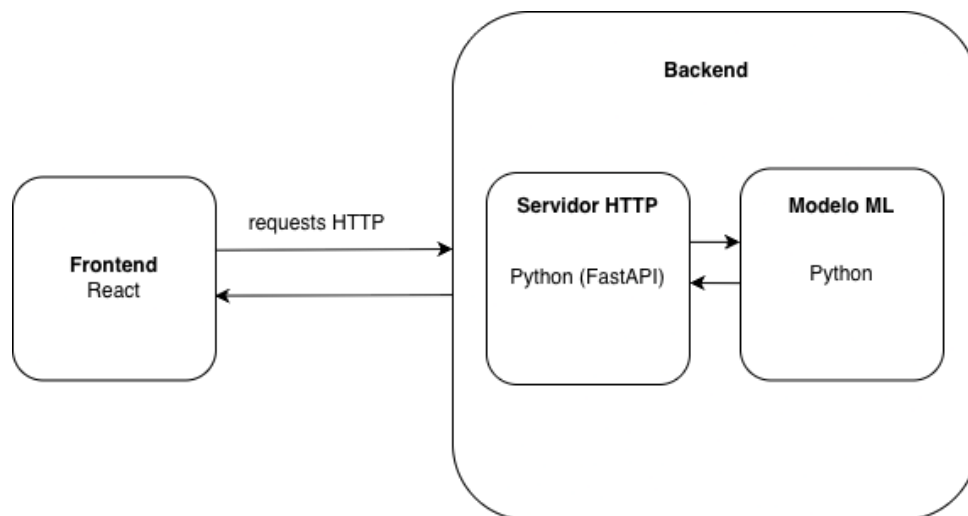


Figura 1: Diagrama de arquitectura del sistema

En las siguientes secciones se detalla cada componente en profundidad.

10.1.1. Aplicación Web

El sistema está compuesto por una arquitectura cliente-servidor, donde el frontend permite la interacción con el usuario y el backend se encarga del procesamiento de datos, autenticación y ejecución del modelo predictivo.

La aplicación permite a los usuarios registrarse e iniciar sesión, gestionando el acceso a las funcionalidades mediante autenticación. Una vez dentro del sistema, el usuario puede cargar diferentes tipos de datos necesarios para la estimación:

- Datos de producción en formato CSV (series temporales).
- Datos PVT (propiedades termodinámicas del fluido).
- Parámetros del reservorio en formato estructurado (por ejemplo, archivos TOML).

Estos datos son procesados por el backend —desarrollado en Python— que integra un modelo de machine learning previamente entrenado. El modelo es capaz de combinar información temporal, física y termodinámica para generar una estimación de la presión del reservorio.

En cuanto a la visualización, el frontend —desarrollado con TypeScript y React— presenta los resultados de manera interactiva y clara para el usuario. Se incluyen gráficos que permiten analizar la evolución de la presión estimada a lo largo del tiempo.

Asimismo, la interfaz incorpora componentes que facilitan la interpretación de los resultados, permitiendo identificar la influencia de las distintas variables de entrada en el comportamiento del modelo. De esta manera, no solo se muestra el valor estimado, sino que también se brinda contexto para su análisis y comprensión.

Además, la aplicación está diseñada con una arquitectura modular que separa claramente las responsabilidades en capas (dominio, infraestructura, repositorios, rutas), facilitando la mantenibilidad y escalabilidad del sistema.

10.1.2. Modelo de Machine Learning

El objetivo del modelo es predecir, a lo largo del tiempo, la presión promedio del reservorio a partir de datos operativos y de las propiedades del fluido.

Para este proyecto utilizamos un modelo de tipo LSTM (Long Short-Term Memory). Para la elección del mismo, se realizaron múltiples pruebas con distintos modelos: regresión lineal, random forests, XG-Boost y series de tiempo. Sin embargo, nos decantamos por LSTM ya que resultó ser el que generaliza de forma más robusta y pareja entre reservorios distintos.

En la sección 13 se detalla el procedimiento de entrenamiento y validación de cada modelo evaluado, junto con la comparación de resultados que motivó la elección del LSTM como modelo final.

Esquema de datos

El esquema de columnas del dataset fue propuesto por nuestros directores con un objetivo concreto: que el modelo tenga a disposición toda la información que aparece en la **ecuación de balance de masa**,

que es la ecuación física que gobierna la evolución de la presión del reservorio. La MBE relaciona la producción acumulada de fluidos, la inyección acumulada, las propiedades termodinámicas del fluido y la geometría del reservorio con el cambio de presión observado en el tiempo.

El dataset consta de dos tablas, cada tabla representa los datos de un reservorio en particular.

La primera (Tabla 1) tiene una fila por timestep e integra la petrofísica y la geometría del reservorio junto con los caudales de producción e inyección.

La segunda (Tabla 2) describe las propiedades termodinámicas del fluido medidas en laboratorio, tabuladas en función de la presión.

Nombre	Símbolo	Unidad	Tipo	Descripción
tiempo_dias	–	–	–	Tiempo en días desde el comienzo de la producción.
Porosidad	ϕ	fracción	Estática (Petrofísica)	Fracción de espacio vacío en la matriz rocosa.
Permeabilidad_mD	k	mD	Estática (Petrofísica)	Capacidad de la roca para permitir el flujo de fluidos.
Espesor_Neto_m	h	m	Estática (Geometría)	Espesor productivo neto de la formación.
Area	A	m ²	Estática (Geometría)	Área total de roca; multiplicada por h se obtiene el volumen total de roca.
Presion_Inicial_Reservorio_psi	P_{init}	psi	Estática (Condición inicial)	Presión promedio del reservorio antes de comenzar la producción.
Caudal_Prod_Petroleo_bbl	q_o	bbl/d	Dinámica (Operativa)	Volumen diario de petróleo extraído en superficie.
Caudal_Prod_Gas_Mpc	Q_g	Mpc/d	Dinámica (Operativa)	Volumen diario de gas producido.
Caudal_Iny_Agua_bbl	q_{inj}	bbl/d	Dinámica (Operativa)	Volumen diario de agua inyectada al reservorio.
Prod_Acumulada_Petroleo	N_p	bbl	Dinámica (Histórica)	Sumatoria histórica del volumen de petróleo extraído.
Prod_Acumulada_Gas	G_p	scf	Dinámica (Histórica)	Sumatoria histórica del volumen de gas extraído.
Prod_Acumulada_Agua	W_p	bbl	Dinámica (Histórica)	Sumatoria histórica del volumen de agua extraída.
Iny_Acumulada_Agua	W_{inj}	bbl	Dinámica (Histórica)	Sumatoria histórica del volumen de agua inyectada.

Tabla 1: Esquema de la tabla de producción e inyección.

Nombre	Símbolo	Unidad	Tipo	Descripción
p_grid_psi	P	psi	Termodinámica (PVT)	Presión a la cual se miden las propiedades termodinámicas.
Bo_rb_stb	B_o	rb/stb	Termodinámica (PVT)	Factor volumétrico del petróleo (relación fondo-superficie).
Bg_rb_scf	B_g	rb/scf	Termodinámica (PVT)	Factor volumétrico del gas (relación fondo-superficie).
Rs_scf_stb	R_s	scf/stb	Termodinámica (PVT)	Relación de gas disuelto por barril de petróleo.

Tabla 2: Esquema de la tabla PVT.

El **target** del modelo es la presión promedio del reservorio en cada timestep (P_r), una columna adicional del primer dataset que no se usa como input sino como variable a predecir.

Generación de datos sintéticos

Para entrenar el modelo LSTM utilizamos datos sintéticos. La razón principal es que conseguir datos reales con el esquema que necesitamos (presión del reservorio en función de variables operativas y propiedades del fluido, a lo largo del tiempo) es difícil sin una colaboración directa con una empresa petrolera, ya que se trata de información sensible que las operadoras manejan bajo confidencialidad.

Para generar los datos sintéticos utilizamos el simulador OPM Flow [9], junto con una herramienta que desarrollamos en Python para poder extraer los datos que necesitamos. El simulador OPM Flow toma como input un modelo de reservorio definido en el formato ECLIPSE [10], que contiene toda la descripción física del campo: la geometría del reservorio, las propiedades de la roca, la tabla PVT del fluido, la configuración de los pozos y el cronograma operativo de producción e inyección. A partir de esa entrada, OPM Flow simula el proceso productivo paso a paso en el tiempo y genera archivos con el historial completo del estado del reservorio, capturando variables como la presión, los caudales de producción e inyección, las saturaciones de cada fase y demás propiedades por celda.

En concreto, utilizamos dos modelos de reservorios que representan campos reales del Mar del Norte y del Mar de Noruega: **Norne** [11] y **Volve** [12]. Decidimos utilizar estos modelos ya que se trata de yacimientos genuinos que estuvieron en producción durante años, operados originalmente por la petrolera noruega Equinor. Además, son dos de los benchmarks más populares en la industria y la academia. La mayoría de los otros modelos públicos disponibles son reservorios teóricos inventados con fines académicos, sin correspondencia con un campo concreto.

Para otorgarle variabilidad al dataset, ejecutamos distintas simulaciones (30 simulaciones del modelo Norne, y 10 simulaciones del modelo Volve), variando ciertos parámetros del reservorio en cada una, de manera de obtener corridas con comportamientos distintos. Las variables que ajustamos fueron la porosidad de la roca, su permeabilidad, la presión inicial del reservorio y el esquema de inyección de

agua. Esto nos permitió obtener un dataset con suficiente diversidad de comportamientos, de modo que el modelo aprenda patrones generalizables en lugar de memorizar una única trayectoria de presión.

A continuación se incluyen visualizaciones de los datasets generados, mostrando variables como la presión, caudales de inyección y producción, en función del tiempo.

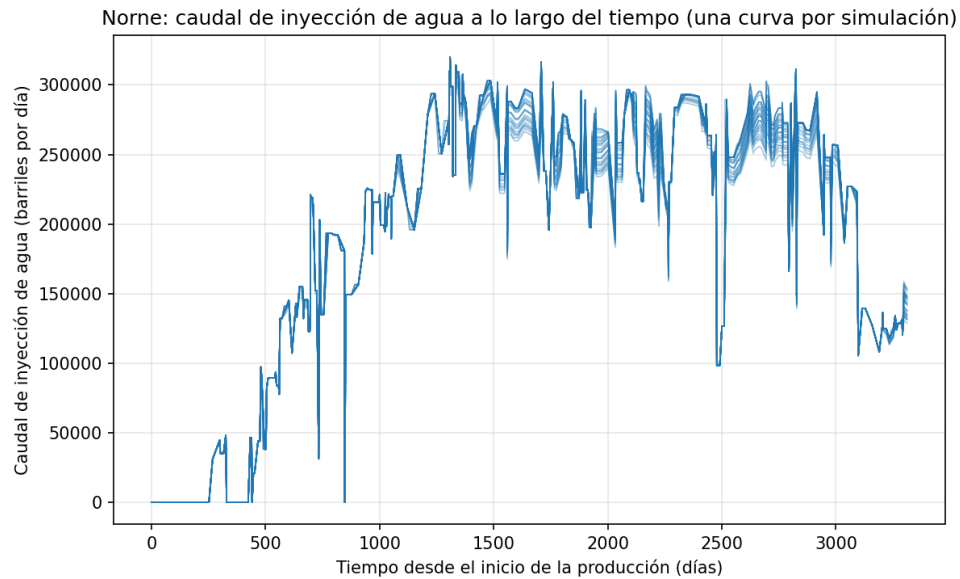


Figura 2: Caudal de inyección de agua (simulaciones Norne)

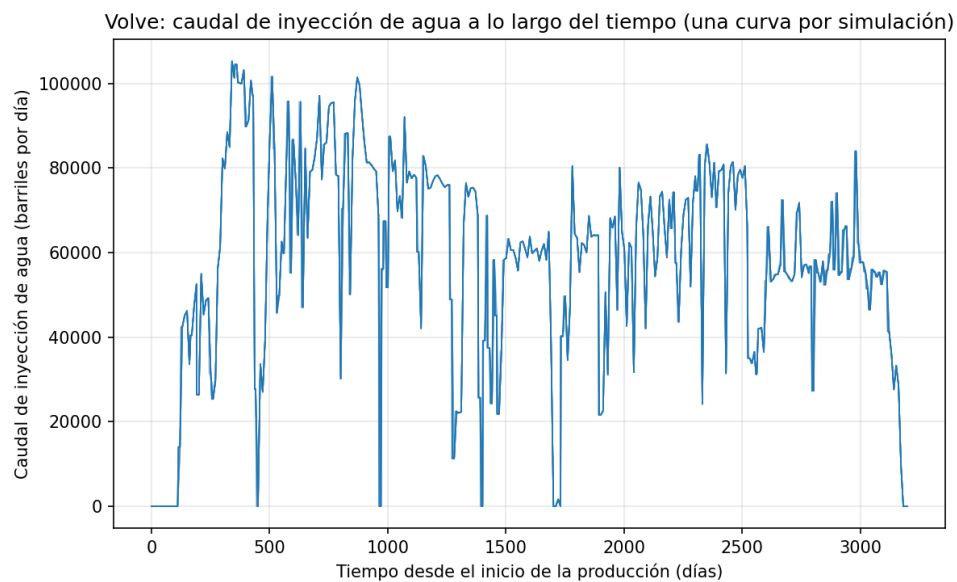


Figura 3: Caudal de inyección de agua (simulaciones Volve)

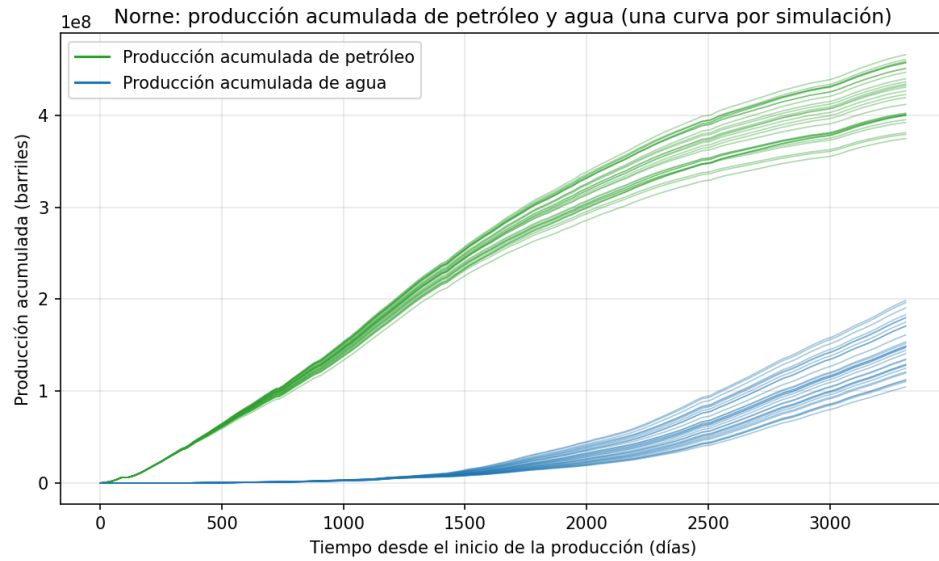


Figura 4: Producción acumulada de petróleo y agua (simulaciones Norne)

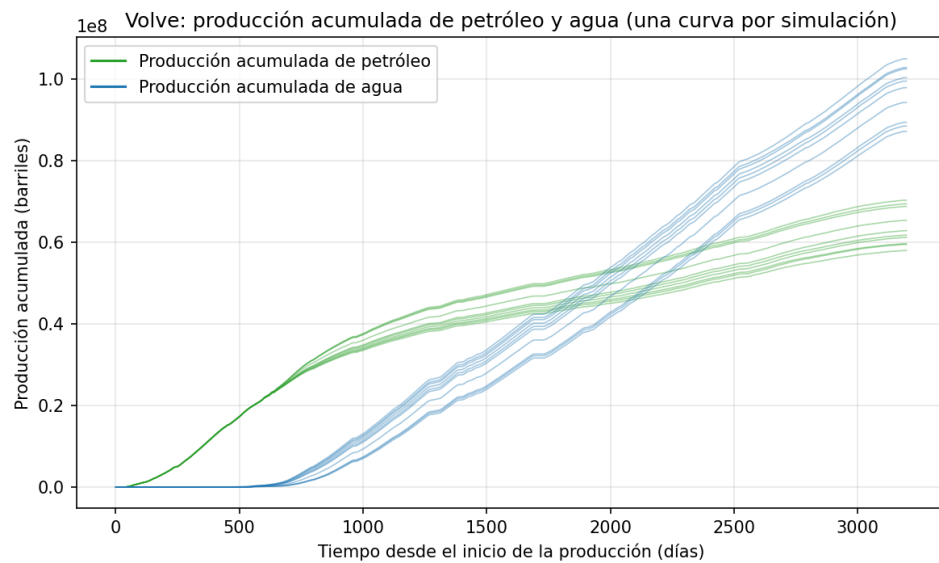


Figura 5: Producción acumulada de petróleo y agua (simulaciones Volve)

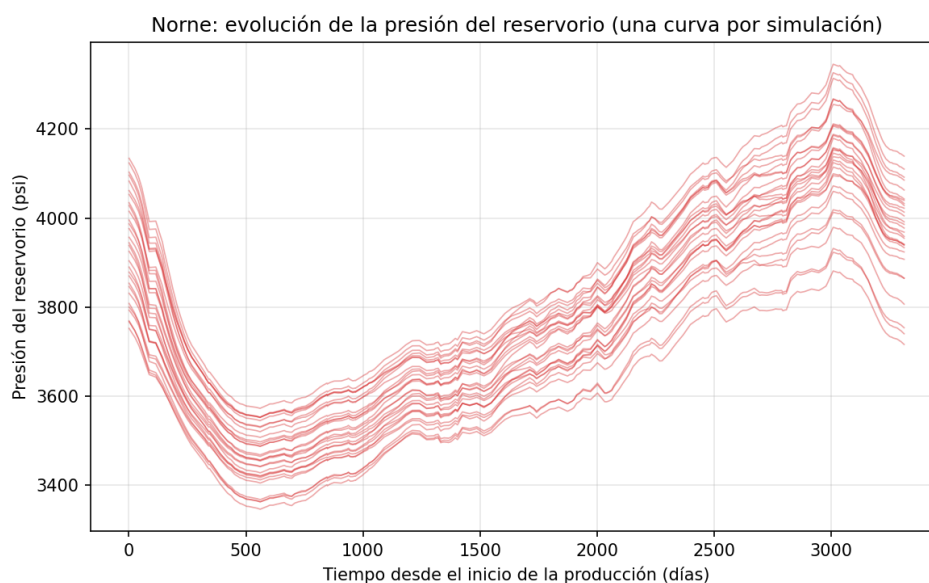


Figura 6: Presión en función del tiempo (simulaciones Norne)

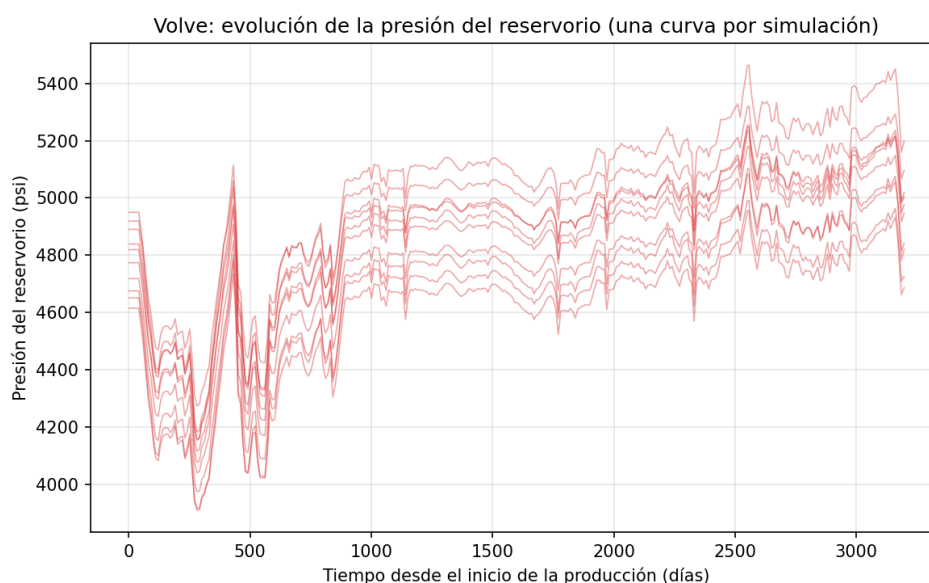


Figura 7: Presión en función del tiempo (simulaciones Volve)

Normalización de datos y Feature Engineering

A partir del análisis exploratorio del dataset, encontramos que los reservorios de Volve y Norne operan en rangos muy distintos de presión, caudales de producción y volúmenes acumulados, lo cual era esperable dado que se trata de campos de tamaños muy diferentes (Norne es aproximadamente 6 veces mas grande que Volve). Esto nos llevó a aplicar varias normalizaciones a los datos antes de pasarlos al modelo.

La primera fue dividir las cantidades dependientes del tamaño del campo (producciones acumuladas,

caudales de producción e inyección) por el pore-volume del reservorio, que es el volumen total que podría llenarse con fluido. De esta forma el modelo ve fracciones del volumen recuperado en lugar de barriles absolutos, y Norne y Volve se vuelven comparables a pesar de las diferencias de escala.

La segunda fue normalizar la presión, es decir, en lugar de predecir el valor absoluto de la presión (que está en el orden de 3000 - 5000 psi), predecimos la diferencia respecto de la presión inicial del reservorio, dividida por una escala fija de 5000 psi. El resultado es un target chico y cercano a cero, más estable para entrenar la red neuronal y que elimina la dependencia con el punto de partida particular de cada corrida.

Además de aplicar normalizaciones, también creamos nuevas features (sugeridas por nuestros directores) a partir de relaciones entre las columnas originales del dataset:

- **Fracciones de fluido recuperado e inyectado:** producciones acumuladas de petróleo y agua e inyección acumulada de agua, divididas por el pore-volume del reservorio. Indican qué proporción del volumen total disponible ya fue extraída o repuesta mediante inyección.
- **Relación gas-petróleo acumulada:** cuánto gas se produjo por cada barril de petróleo hasta el momento.
- **Relación agua-petróleo instantánea:** cuántos barriles de agua se producen por cada barril de petróleo en ese instante. Es un indicador clásico del avance del frente de agua hacia los pozos productores.
- **Water cut acumulado:** porcentaje del total de líquidos producidos que es agua.
- **Voidage replacement ratio simplificado** [13]: relación entre el volumen total inyectado y el volumen total producido. Indica si la inyección de agua compensa la extracción y si se está logrando mantener la presión del reservorio.

Un problema con el que nos encontramos fue cómo incluir la tabla PVT como input del modelo, dado que es una tabla separada del dataset principal y con una estructura distinta a la de la información temporal, ya que tiene una fila por nivel de presión (con los valores de B_o , B_g y R_s medidos en laboratorio para ese nivel) y no cambia en el tiempo. Lo resolvimos agregando un **encoder de PVT**, un módulo pequeño que se entrena junto con el resto del modelo y cuya función es comprimir toda la tabla PVT a un vector resumen compacto. Ese vector se le pasa al modelo principal como un dato adicional por simulación, de manera que la información termodinámica del fluido influya en la predicción sin opacar a las variables que cambian con el tiempo.

11 Metodología aplicada

El trabajo se organizó de forma incremental, en ciclos marcados por las reuniones de seguimiento con los directores. Cada ciclo apuntaba a un objetivo concreto y se revisaba con ellos antes de pasar al siguiente.

11.1 Desarrollo incremental y entregables

El desarrollo estuvo condicionado por la disponibilidad de los datos. La etapa inicial se dedicó a estudiar el dominio, ya que el equipo tenía poca formación previa en ingeniería de reservorios, y a escribir la propuesta, mientras se gestionaba el acceso a datos reales de presión de la industria. Esos datos nunca llegaron. Ante eso se optó por datos sintéticos: primero generados con modelos de lenguaje y, como no daban suficiente realismo físico, después con el simulador OPM Flow, que terminó siendo la fuente de datos del proyecto. Recién a partir de ahí el trabajo se centró en probar y comparar modelos (baseline lineal, Random Forest, XGBoost y LSTM), construir el MVP de la aplicación web e iterar su experiencia de usuario con los directores, para cerrar con este informe.

Hubo tres entregas formales: la propuesta de anteproyecto, una entrega intermedia a mitad de proyecto (un video con el estado del trabajo) y este informe final. Entre medio, los avances importantes, como los datasets generados o el flujo de carga de la aplicación, se mostraban a los directores en reuniones específicas. El seguimiento era quincenal: ahí se revisaban los avances y se definían los próximos pasos.

11.2 Control de versiones

Se usó Git para versionar el código y GitHub como plataforma, con los repositorios agrupados en una organización del equipo. El código se separó por repositorio: uno para los experimentos y notebooks, otro para la aplicación web y otro para el informe en $\text{\LaTeX}$. Para los desarrollos grandes se trabajó con ramas y *pull requests*; los cambios chicos iban directo a la rama principal. Antes de integrar, los cambios se revisaban entre pares, tanto el código de la aplicación como los notebooks. Hacia el final esa revisión se apoyó en el CI, que frena cualquier cambio que rompa las pruebas.

11.3 Gestión del proceso y seguimiento

Las tareas y los bugs se siguieron como *issues* en GitHub, complementados con una lista de pendientes compartida y la coordinación diaria por mensajería. Con los directores y demás interesados la comunicación fue por mensajería y videollamadas.

11.4 Automatización

La aplicación web tiene un pipeline de integración continua (CI) con GitHub Actions que corre en cada *push* y *pull request*. Ejecuta los tests del backend con *pytest*, exigiendo un mínimo de cobertura, y compila el frontend chequeando tipos. Si el pipeline falla, el cambio no se puede integrar. No hay despliegue automático: por ahora la aplicación corre en entorno local.

11.5 Criterios de aceptación

Un modelo se daba por bueno cuando alcanzaba métricas razonables en las dos evaluaciones, *in-distribution* y *cross-reservoir*: un R^2 positivo (con 0,5 como referencia) y un MAE dentro del rango físico esperado. A eso se sumaba el visto bueno de los directores. En la aplicación web, una funcionalidad se consideraba terminada cuando funcionaba de extremo a extremo (carga de datos, predicción y visualización), pasaba el CI y se validaba en las reuniones.

11.6 Artefactos de gestión y comunicación

La comunicación con los interesados se apoyó en minutas de las reuniones, donde quedaban registradas las decisiones y los próximos pasos. El alcance se manejó a partir del documento de propuesta inicial. Los riesgos se tratan en la sección correspondiente de este informe.

11.7 Indicadores de calidad

La calidad del producto se midió con las métricas de los modelos (R^2 y MAE) y con el estado del CI: pruebas en verde y un mínimo de cobertura. Del lado del proceso, se siguió que se mantuviera la cadencia de reuniones y que cada cambio pasara por revisión antes de integrarse.

El avance del proyecto se midió contra los hitos formales (anteproyecto, entrega intermedia e informe final) y por lo que se completaba en cada iteración: datasets generados, modelos evaluados y funcionalidades de la aplicación.

12 Herramientas externas utilizadas

Aplicación Web

Para el desarrollo de la aplicación web se utilizaron diversas herramientas y tecnologías, tanto para el backend como para el frontend y el despliegue del sistema:

- **FastAPI**: framework web en Python utilizado para la construcción de la API REST, permitiendo un desarrollo rápido y eficiente de endpoints.
- **NumPy y Pandas**: utilizadas para la manipulación y procesamiento de datos, especialmente en la carga y transformación de archivos CSV.
- **Alembic**: herramienta de migraciones de base de datos, utilizada para gestionar cambios en el esquema de la base de datos.
- **SQLAlchemy**: ORM utilizado para la interacción con la base de datos desde el backend.
- **JWT (JSON Web Tokens)**: utilizado para la autenticación y autorización de usuarios dentro del sistema.
- **React**: biblioteca de JavaScript utilizada para la construcción de la interfaz de usuario.
- **TypeScript**: lenguaje utilizado en el frontend para mejorar la calidad del código mediante tipado estático.
- **Vite**: herramienta de desarrollo utilizada para el bundling y ejecución del frontend.
- **Charting libraries (gráficos)**: utilizadas para la visualización de resultados, incluyendo gráficos de presión y análisis de variables.
- **Docker y Docker Compose**: utilizados para contenerizar la aplicación y facilitar su despliegue en diferentes entornos.
- **Git**: sistema de control de versiones utilizado para la gestión del código fuente.

Estas herramientas permitieron construir una solución robusta, escalable y mantenible, integrando tanto capacidades de procesamiento de datos como visualización e interacción con el usuario.

Modelo de Machine Learning

Para el desarrollo del modelo de machine learning se utilizaron distintas herramientas y librerías, tanto para el procesamiento de los datos como para el entrenamiento, evaluación y visualización de los resultados:

- **Python:** lenguaje principal utilizado para el entrenamiento y evaluación de los modelos, en conjunto con las siguientes librerías:
 - **PyTorch:** para el entrenamiento y evaluación del modelo LSTM.
 - **scikit-learn:** para el entrenamiento y evaluación de los modelos de regresión, random forest, y XGBoost.
 - **NumPy y Pandas:** utilizadas para la manipulación y procesamiento de los datasets en formato CSV.
 - **Matplotlib:** utilizada crear visualizaciones, tanto para el análisis exploratorio como para los resultados de los distintos modelos.
 - **resdata** [14]: librería de Python utilizada para parsear los archivos de salida de OPM Flow y generar el dataset sintético.
- **Google Colab:** entorno de ejecución utilizado para correr los notebooks de entrenamiento y evaluación.
- **Jupyter notebooks:** utilizados para ejecutar los notebooks de forma automatizada y reproducible.
- **OPM Flow:** simulador open source de flujo en medios porosos utilizado para generar el dataset sintético a partir de los modelos de reservorio en formato ECLIPSE.
- **Docker:** utilizado para correr OPM Flow en contenedores independientes y paralelizar la generación del dataset sin conflictos entre simulaciones concurrentes.

13 Experimentación y/o validación

Durante el entrenamiento de los modelos se realizaron múltiples pruebas para validar la efectividad de los mismos, compararlos, y decidir cuál utilizar en el proyecto:

Dado que contamos con dos modelos de reservorio (Norne y Volve), esto nos permitió validar que el modelo es capaz de generalizar entre reservorios distintos. En este contexto, realizamos dos tipos de pruebas:

- **Evaluación in-distribution:** el modelo se entrena con 24 de las 30 simulaciones de Norne y se evalúa sobre las 6 simulaciones restantes de Norne. Esto nos permite medir qué tan bien el modelo predice la presión del campo con el que fue entrenado.
- **Evaluación cross-reservoir:** el modelo se evalúa sobre las 10 simulaciones completas de Volve, un reservorio al que nunca fue expuesto durante el entrenamiento. Esto nos permite saber si el modelo aprendió la física subyacente o si solamente memorizó la distribución particular de Norne.

Para definir la calidad de los modelos, nos basamos en las siguientes métricas:

- **R² (coeficiente de determinación):** mide qué fracción de la varianza del target explica el modelo.
- **MAE (Mean Absolute Error):** el error promedio absoluto en la misma unidad física que la presión del reservorio (psi).

13.1 Evaluación y análisis comparativo de los modelos de machine learning

A continuación se presenta un análisis comparativo de todos los modelos que se probaron durante la fase de entrenamiento.

13.1.1. Modelo baseline

El primer modelo que probamos fue una regresión lineal con regularización Ridge sobre las features adimensionales del reservorio. Con este modelo no buscamos alcanzar buena performance, sino establecer una cota inferior empírica contra la cual comparar los modelos posteriores.

Split	R ²	MAE (psi)
Norne test (in-distribution)	0.959	35.23
Volve (cross-reservoir)	-891.83	8166.51

Tabla 3: Métricas del modelo baseline (regresión Ridge).

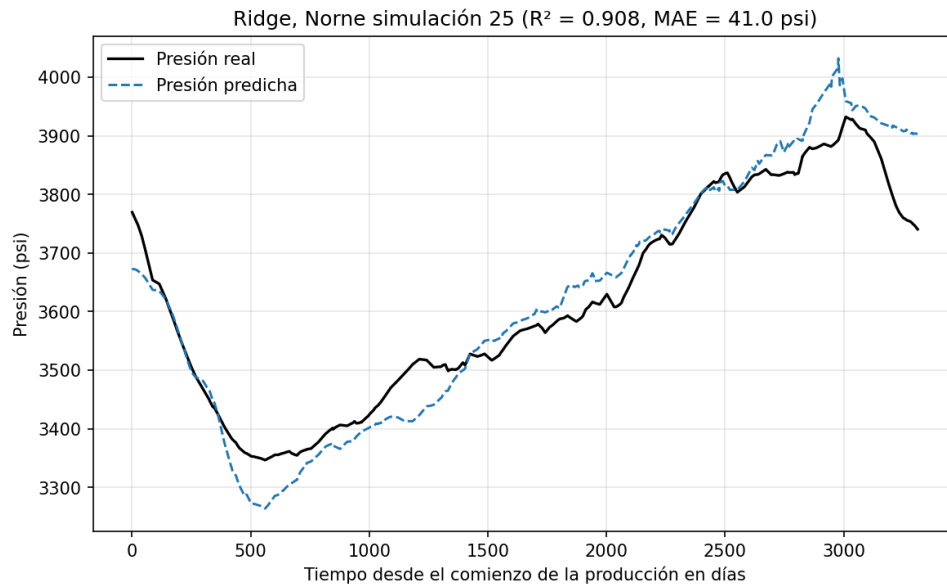


Figura 8: Comparación entre la presión real de Norne y la presión predicha por el modelo baseline

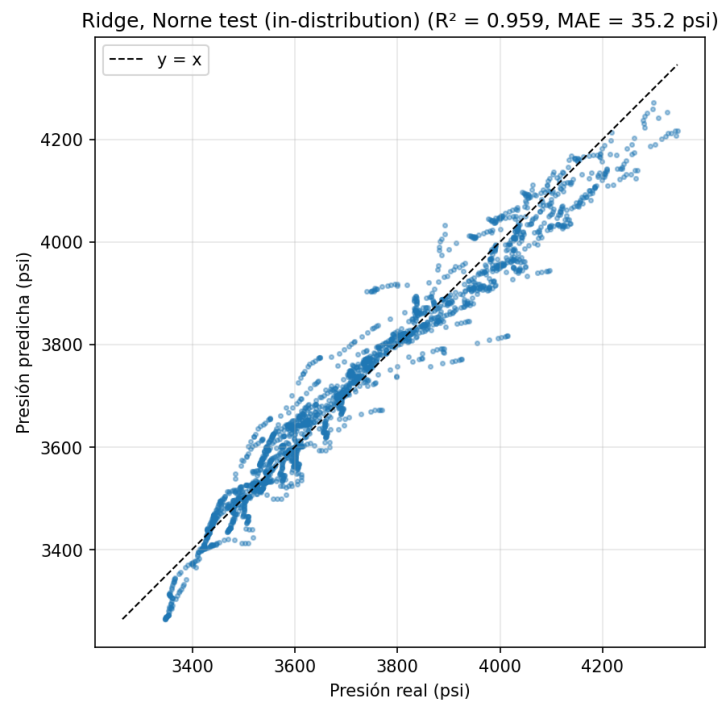


Figura 9: Parity plot entre la presión real de Norne vs. la presión predicha por el modelo baseline

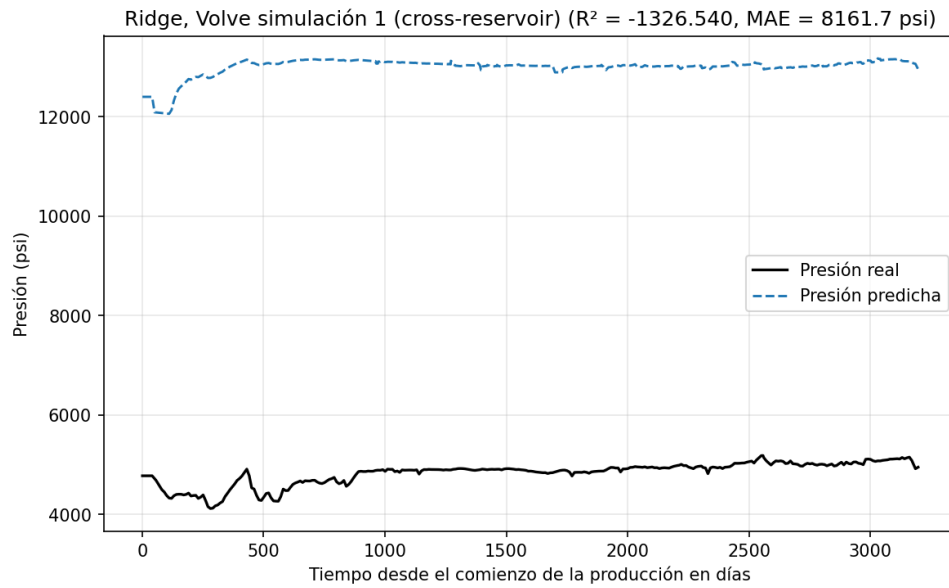


Figura 10: Comparación entre la presión real de Volve y la presión predicha por el modelo baseline

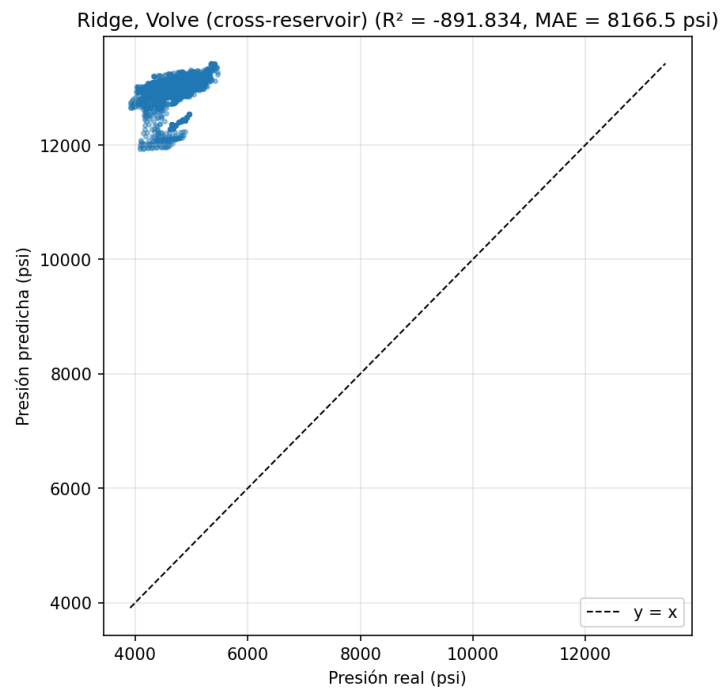


Figura 11: Parity plot entre la presión real de Volve vs. la presión predicha por el modelo baseline

A partir de los valores del coeficiente de determinación (0.959 para Norne, -891.83 para Volve), se puede deducir que claramente el modelo overfittea para los datos de entrenamiento, y no logra extrapolar la predicción para reservorios con una distribución distinta a la de Norne.

13.1.2. Random Forest

Una vez realizadas las pruebas con un modelo baseline, procedimos a entrenar un modelo Random Forest, con el objetivo de capturar interacciones no lineales entre las features que la regresión lineal no podía representar.

Tabla 4: Métricas del modelo Random Forest.

Split	R ²	MAE (psi)
Norne test (in-distribution)	0.977	22.99
Volve (cross-reservoir)	-9.03	842.43

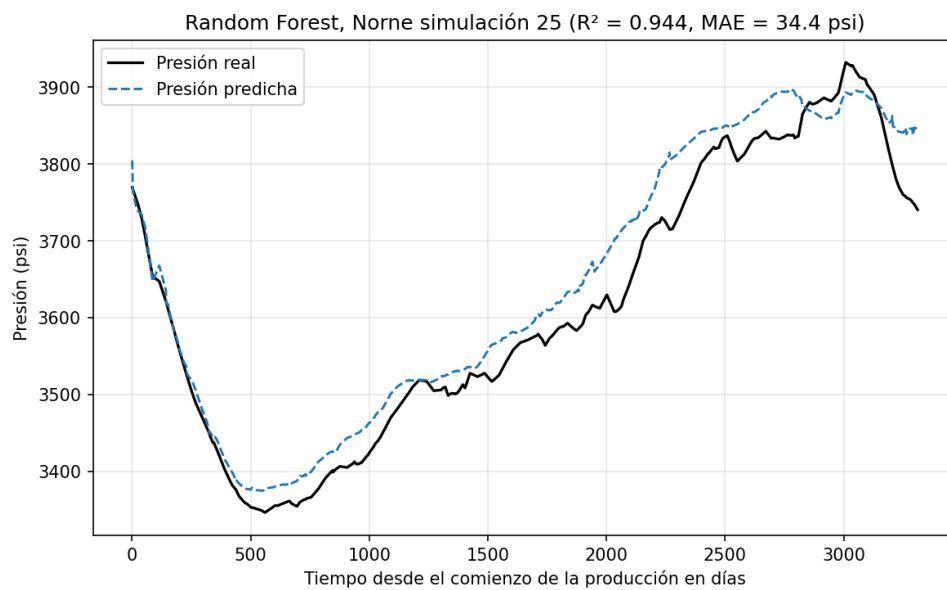


Figura 12: Comparación entre la presión real de Norne y la presión predicha por el modelo random forest

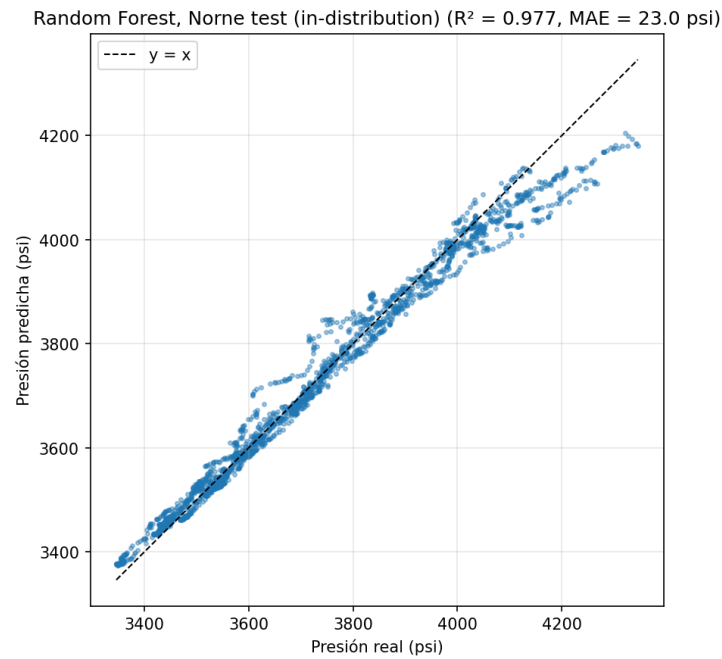


Figura 13: Parity plot entre la presión real de Norne vs. la presión predicha por el modelo random forest

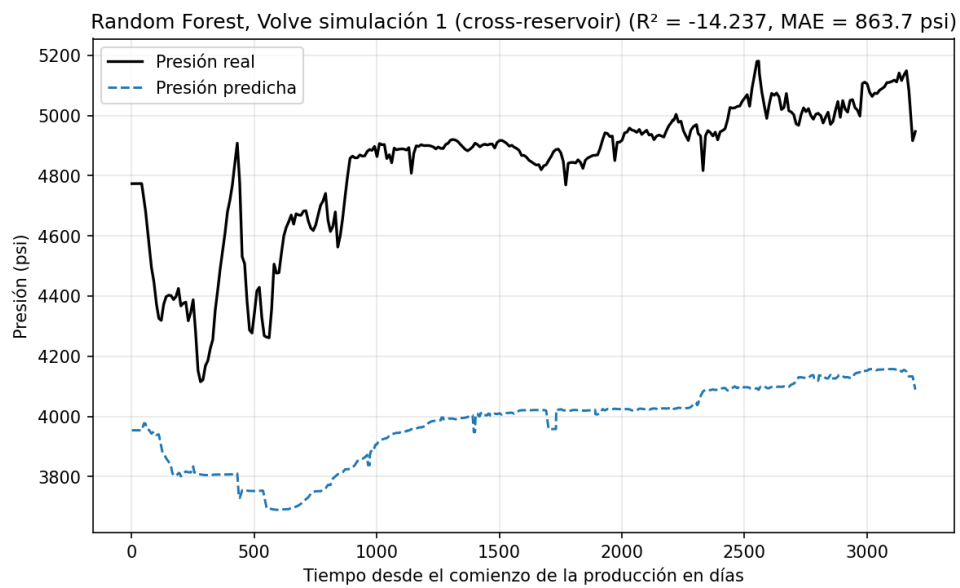


Figura 14: Comparación entre la presión real de Volve y la presión predicha por el modelo random forest

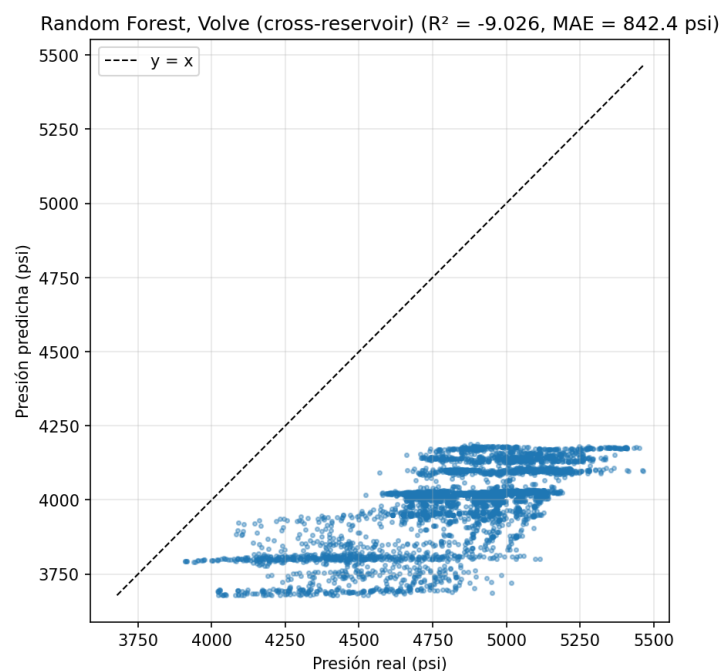


Figura 15: Parity plot entre la presión real de Volve vs. la presión predicha por el modelo random forest

Random Forest presenta una mejora respecto al modelo baseline sobre Norne: el R^2 sube de 0.96 a 0.98 y el MAE baja de 35 a 23 psi. Sin embargo, el modelo no logra extrapolar estos resultados al reservorio Volve, el R^2 es de -9.03 y el MAE supera los 800 psi. Aunque las predicciones ya no son tan extremas como las del baseline lineal, la incapacidad de generalizar fuera de la distribución de entrenamiento se mantiene.

13.1.3. XGBoost

El siguiente modelo a evaluar es XGBoost, es un modelo de árboles un poco más complejo, que en general supera a Random Forest en problemas tabulares.

Tabla 5: Métricas del modelo XGBoost.

Split	R^2	MAE (psi)
Norne test (in-distribution)	0.986	19.80
Volve (cross-reservoir)	-9.93	864.89

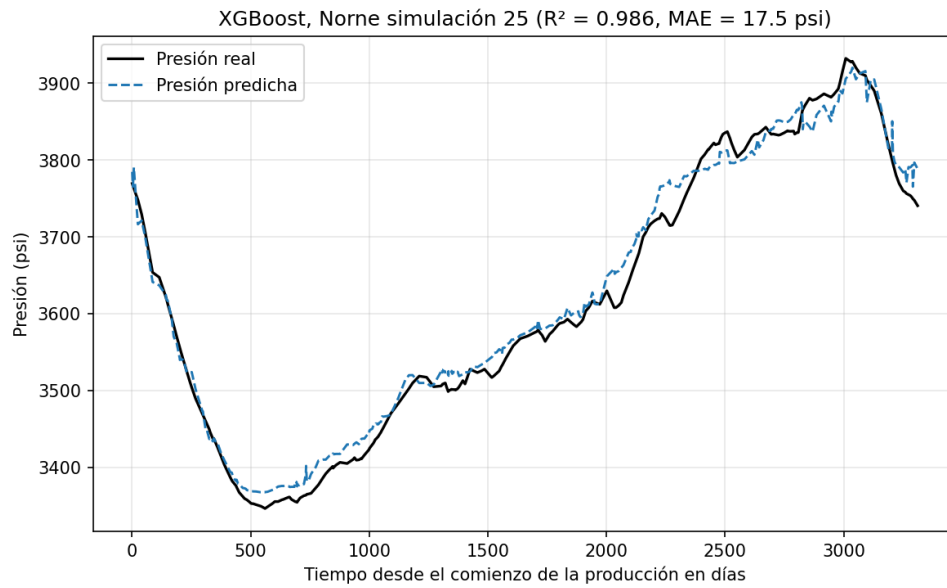


Figura 16: Comparación entre la presión real de Norne y la presión predicha por el modelo XGBoost

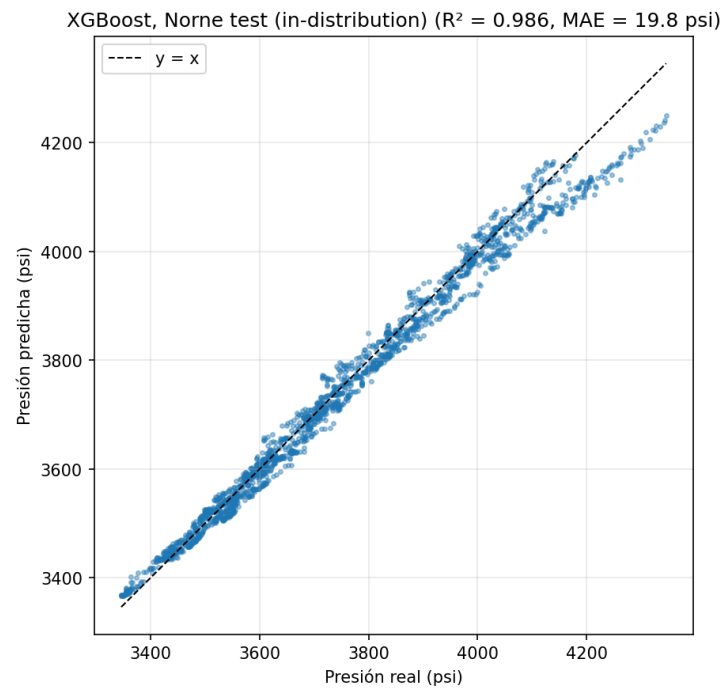


Figura 17: Parity plot entre la presión real de Norne vs. la presión predicha por el modelo XGBoost

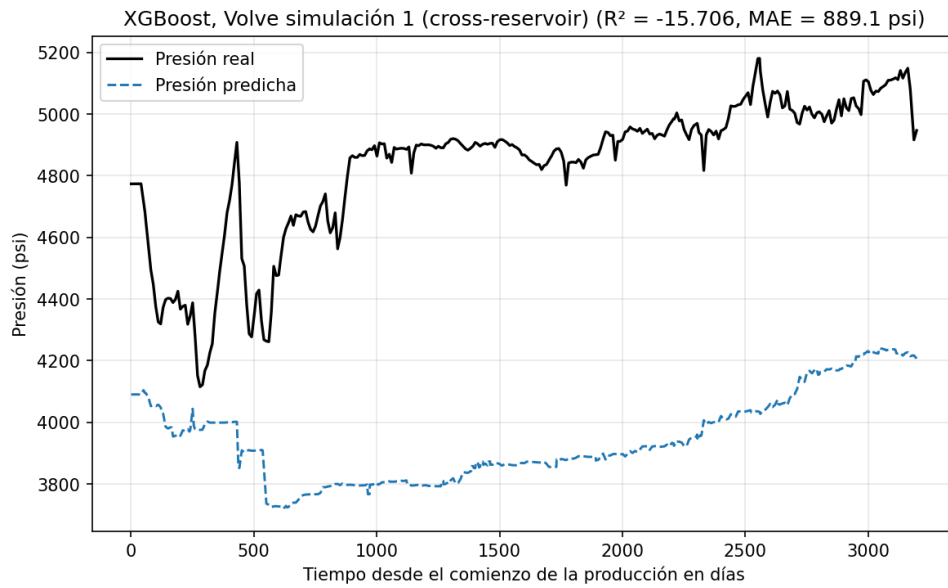


Figura 18: Comparación entre la presión real de Volve y la presión predicha por el modelo XGBoost

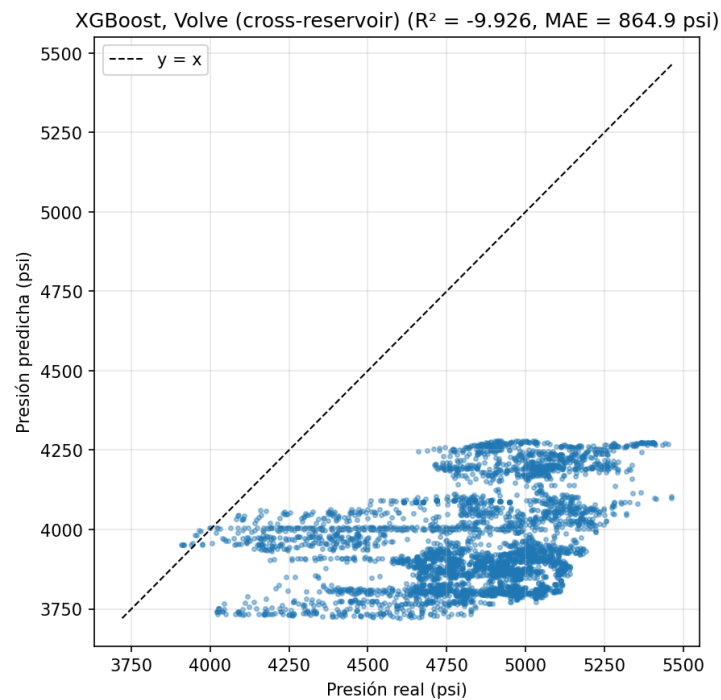


Figura 19: Parity plot entre la presión real de Volve vs. la presión predicha por el modelo XGBoost

XGBoost obtiene la mejor performance in-distribution sobre Norne de los modelos probados: $R^2 = 0.986$ y MAE de apenas 19.80 psi, valores muy superiores al baseline. Sin embargo, mantiene la misma incapacidad de generalizar entre reservorios: el R^2 sobre Volve es de -9.93 y el MAE de 865 psi.

13.1.4. LSTM

En vista de que los modelos lineales y de árboles no son capaces de aprender la física subyacente del problema, decidimos probar un modelo LSTM, que suele tener buen rendimiento sobre datos basados en series de tiempo.

Tabla 6: Métricas del modelo LSTM

Split	R ²	MAE (psi)
Norne test (in-distribution)	0.910	51.70
Volve (cross-reservoir)	0.735	101.90

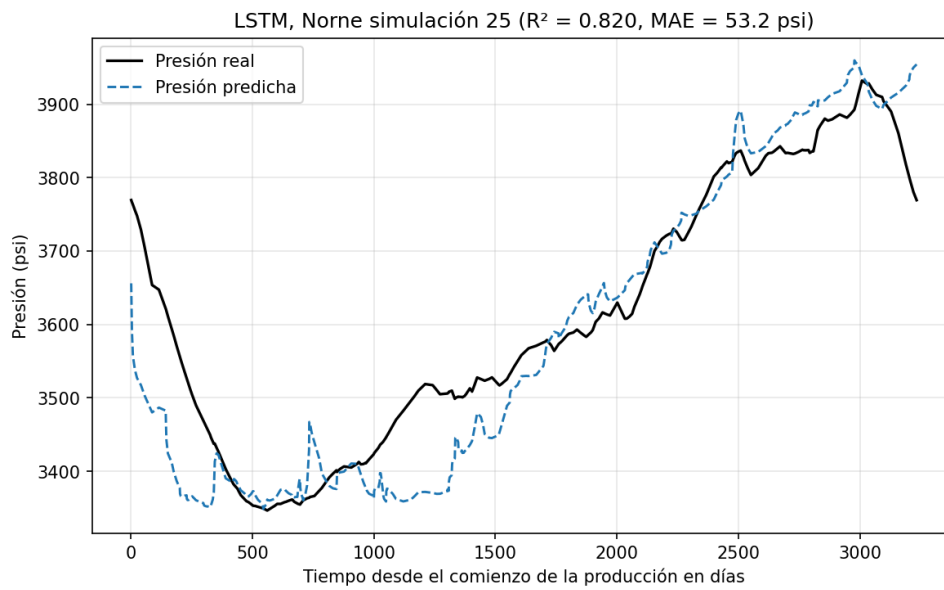


Figura 20: Comparación entre la presión real de Norne y la presión predicha por el modelo LSTM

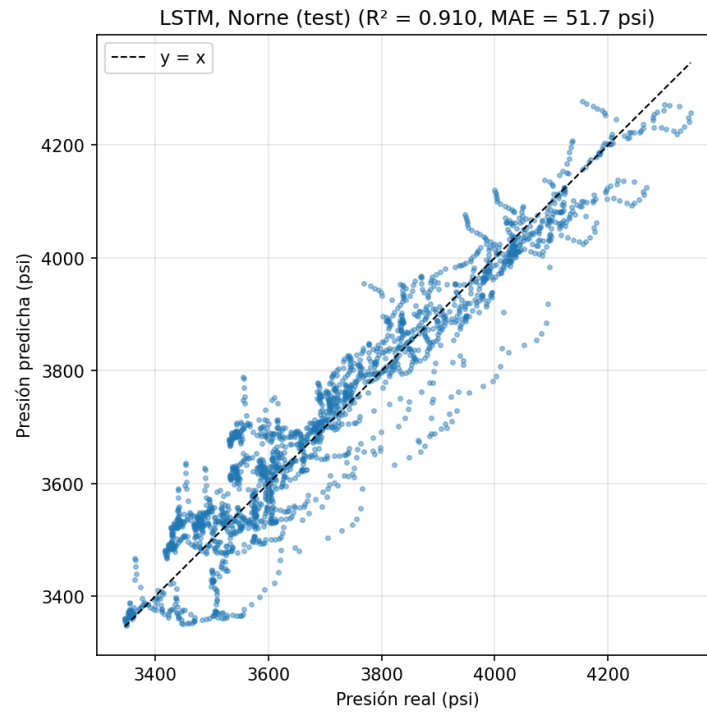


Figura 21: Parity plot entre la presión real de Norne vs. la presión predicha por el modelo LSTM

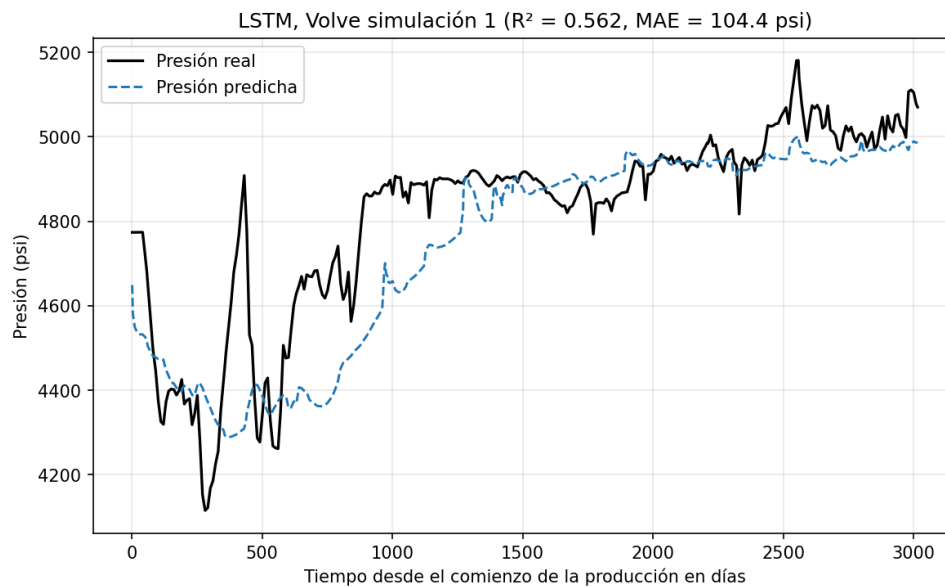


Figura 22: Comparación entre la presión real de Volve y la presión predicha por el modelo LSTM

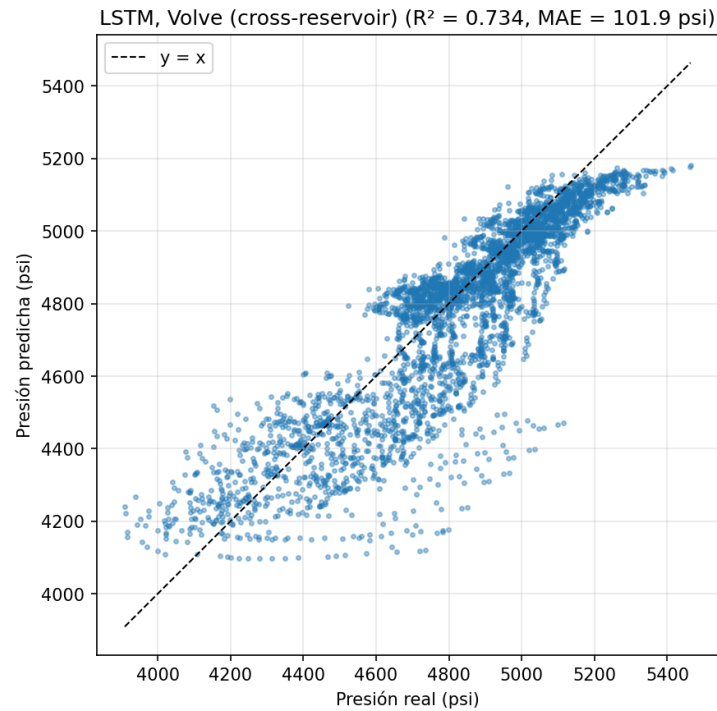


Figura 23: Parity plot entre la presión real de Volve vs. la presión predicha por el modelo LSTM

A diferencia de los otros modelos, podemos ver que LSTM logra buenos resultados tanto para el reservorio Norne como para Volve, es decir, es capaz de generalizar la predicción entre reservorios distintos.

En Norne, el modelo alcanza un R^2 de 0.91 en el test in-distribution, un valor algo más bajo que XGBoost en su propio reservorio (0.99). Sin embargo, el resultado más interesante está en los resultados de las predicciones de Volve, ya que se consigue un R^2 de 0.735 con un MAE de 101.9 psi, mientras que los demás modelos, bajo este protocolo, se mantenían en valores de R^2 negativos sobre Volve.

El modelo LSTM logra generar predicciones que mantienen la forma general de la trayectoria de presión de cada simulación de Volve, en contraste con los modelos lineales y de árboles, que predicen valores muy fuera del rango físico real.

13.1.5. Comparación final entre modelos

Los gráficos 24 y 25 comparan visualmente las métricas de los cuatro modelos sobre los dos splits. Dado que todos fueron entrenados con el mismo protocolo (24 simulaciones de Norne como train, 6 como test, las 10 simulaciones de Volve como evaluación cross-reservoir), la comparación es directa.

El gráfico de MAE usa escala logarítmica porque el rango de errores cubre tres órdenes de magnitud entre el mejor caso (XGBoost en Norne, 19.8 psi) y el peor (Ridge en Volve, más de 8000 psi). En el gráfico de R^2 , el valor del baseline en Volve ($-891,83$) está truncado para no ofuscar al resto de la escala.

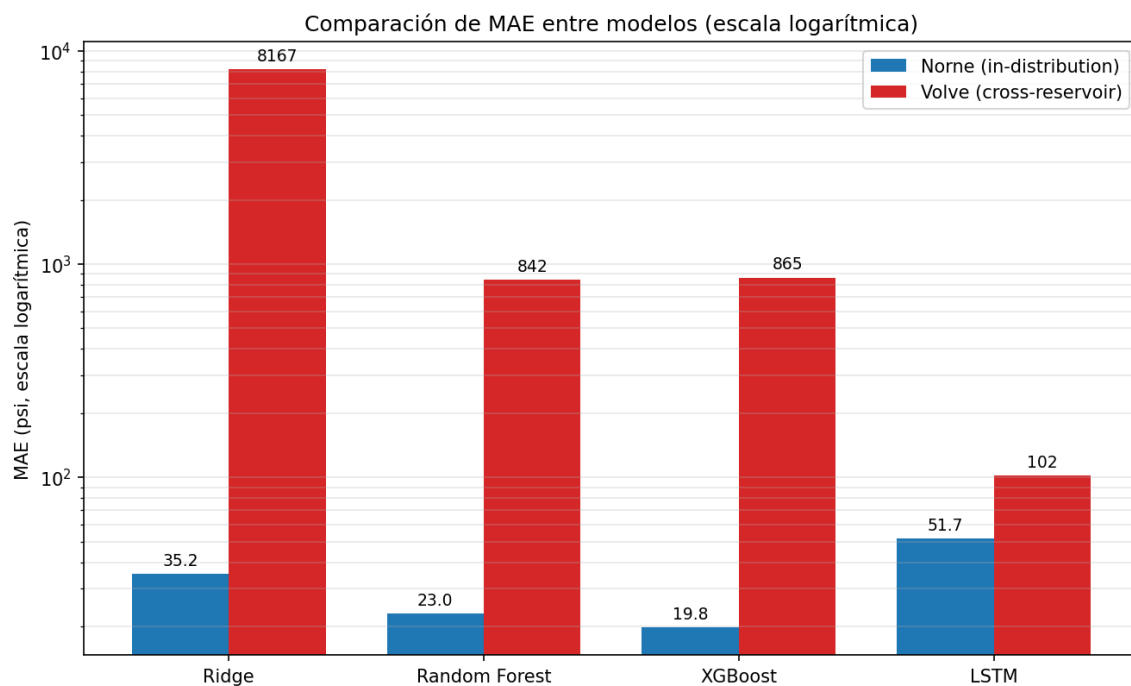


Figura 24: Comparación del MAE entre los cuatro modelos, en Norne (in-distribution) y Volve (cross-reservoir). Escala logarítmica en el eje vertical.

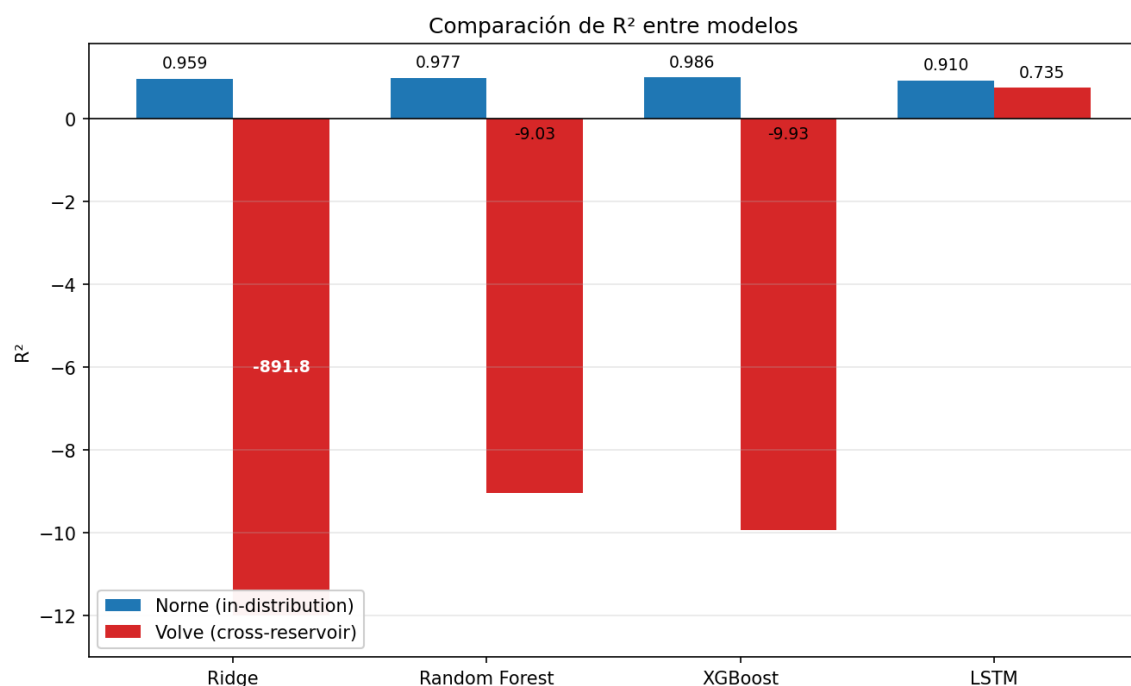


Figura 25: Comparación del R^2 entre los cuatro modelos.

A partir de los gráficos se ve con claridad lo que ya habíamos observado en los análisis individuales de cada modelo: bajo este protocolo, el LSTM es el que mejor generaliza al reservorio no visto. Los demás modelos alcanzan métricas muy buenas in-distribution sobre Norne pero colapsan al evaluar

sobre Volve, mientras que el LSTM mantiene un R^2 positivo y un MAE varios órdenes de magnitud menor en el split cross-reservoir.

Ahora bien, este colapso de los modelos lineales y de árboles sobre Volve depende fuertemente de la formulación del problema y no debe leerse como una incapacidad intrínseca de esas familias de modelo. En este experimento, el LSTM predice la diferencia de presión respecto de P_{init} (un objetivo anclado y transferible entre campos), mientras que los modelos restantes fueron entrenados para predecir la presión absoluta. Como se muestra en la sección 13.2, cuando los modelos de árboles reciben el mismo anclaje a P_{init} , también logran transferir a un campo no visto. En otras palabras, el factor determinante de la transferencia resulta ser el anclaje a P_{init} y la cobertura de dominio, más que la familia de modelo elegida.

13.2 Validación de generalización multi-campo (leave-one-field-out)

La comparación anterior entrena con un único campo (Norne) y evalúa la transferencia a un segundo campo (Volve). Para poner a prueba la generalización de forma más exigente, realizamos un segundo experimento que incorpora un tercer reservorio, **SPE9** [15], y adopta un protocolo de *leave-one-field-out*: para cada campo objetivo, el modelo se entrena con los otros dos campos completos y predice el campo que nunca vio.

Los tres campos utilizados son Volve (10 simulaciones), Norne (30) y SPE9 (100). Para que la comparación entre familias de modelo sea justa, los tres reciben exactamente la misma entrada: el mismo conjunto de features adimensionales (normalizadas por el pore-volume, sin lags del target y descartando B_o , B_g y R_s por su dependencia directa de la presión), y el mismo objetivo anclado a la presión inicial ($\Delta = P - P_{init}$, del que se reconstruye $P = P_{init} + \hat{\Delta}$). Anclar a P_{init} es lo que permite comparar campos con niveles de presión distintos: sin este anclaje, los tres modelos arrojan R^2 negativo en todos los campos.

Se comparan tres familias: Ridge (lineal regularizado), XGBoost (árboles con boosting) y LSTM (red recurrente). Cabe aclarar que el LSTM evaluado en este experimento es una versión genérica sobre las mismas features, que no incorpora el encoder de PVT descrito anteriormente; por lo tanto, sus métricas aquí constituyen una cota inferior de su rendimiento real.

Campo objetivo (R^2 zero-shot)	Ridge	XGBoost	LSTM
Volve	-0.68	+0.69	+0.53
Norne	+0.80	+0.23	+0.72
SPE9	+0.38	-1.21	-0.86
R^2 medio	+0.17	-0.10	+0.13
R^2 mediano por simulación	-0.16	-0.33	-0.05

Tabla 7: Generalización entre campos (leave-one-field-out). Cada fila es un campo no visto durante el entrenamiento. El R^2 mediano por simulación es más exigente: obliga a seguir la trayectoria de cada corrida, no sólo el nivel promedio del campo.

De este experimento se desprenden varias observaciones:

- **No hay un ganador único entre familias de modelo.** Cada modelo gana en un campo distinto: XGBoost transfiere mejor a Volve ($R^2 = 0,69$) y Ridge a Norne ($R^2 = 0,80$). A diferencia de lo observado en el experimento anterior, los modelos de árboles *sí* logran transferir a un campo no visto cuando reciben el target anclado a P_{init} .
- **El LSTM es el más robusto.** Es el único positivo en Volve y Norne a la vez y obtiene el mejor R^2 mediano por simulación; no se desploma como XGBoost en el campo más difícil. El costo es un entrenamiento sensiblemente más lento.
- **XGBoost es el más volátil.** Su mayor capacidad le da el mejor resultado en Volve pero el peor en SPE9 ($-1,21$): cuando el campo nuevo es muy distinto, sobreajusta y extrapola mal.
- **Ningún modelo resuelve SPE9.** Es el campo más distinto de los tres —su presión cruza el punto de burbuja y libera gas, un régimen ausente en el entrenamiento— y los tres modelos dan un R^2 pobre o negativo. El $+0,38$ de Ridge es engañoso: su R^2 por simulación es prácticamente nulo, es decir, acierta el nivel promedio entre simulaciones pero no la trayectoria real.

La conclusión central de este experimento es que el factor que determina la transferencia no es tanto la familia de modelo (Ridge vs. XGBoost vs. LSTM) sino dos elementos metodológicos: **anclar la predicción a P_{init}** y **contar con cobertura de dominio**, esto es, que el entrenamiento incluya algún campo parecido al campo objetivo. Los modelos transfieren bien cuando el campo nuevo se asemeja a los de entrenamiento (Volve, Norne) y fallan cuando pertenece a un régimen que nunca vieron (SPE9, que cruza el punto de burbuja).

14 Cronograma de las actividades realizadas

El desarrollo del proyecto se llevó a cabo a lo largo de dos cuatrimestres, cada uno con objetivos y actividades específicas, en línea con la dedicación y duración establecidas para la materia.

La planificación contempló una dedicación aproximada de 12 horas semanales por estudiante durante 32 semanas, alcanzando un total estimado de 384 horas de trabajo. Esta organización permitió abordar el proyecto de manera progresiva, incluyendo tanto el desarrollo técnico como la elaboración del informe final y la preparación de la exposición.

A lo largo de la cursada, se mantuvo un seguimiento periódico mediante reuniones quincenales con el tutor, en las cuales se evaluaron los avances realizados, se resolvieron dudas especialmente vinculadas al dominio del problema y se definieron los próximos pasos a seguir.

A continuación, se presenta el cronograma detallado de las actividades desarrolladas en cada etapa del proyecto.

14.1 Cronograma del segundo cuatrimestre 2025

A continuación, se presenta el cronograma detallado de las actividades desarrolladas durante el segundo cuatrimestre de 2025, comprendido entre los meses de agosto y diciembre, siendo un total de 16 semanas.

Durante este período se llevaron a cabo tareas clave vinculadas a la definición de la propuesta del proyecto, el estudio del dominio de aplicación, la identificación de variables relevantes y la espera de datos provenientes de la industria necesarios para el desarrollo del modelo de machine learning.

El diagrama de Gantt correspondiente se encuentra organizado en unidades mensuales, lo que permite una visualización clara de la distribución temporal de las actividades a lo largo del cuatrimestre.

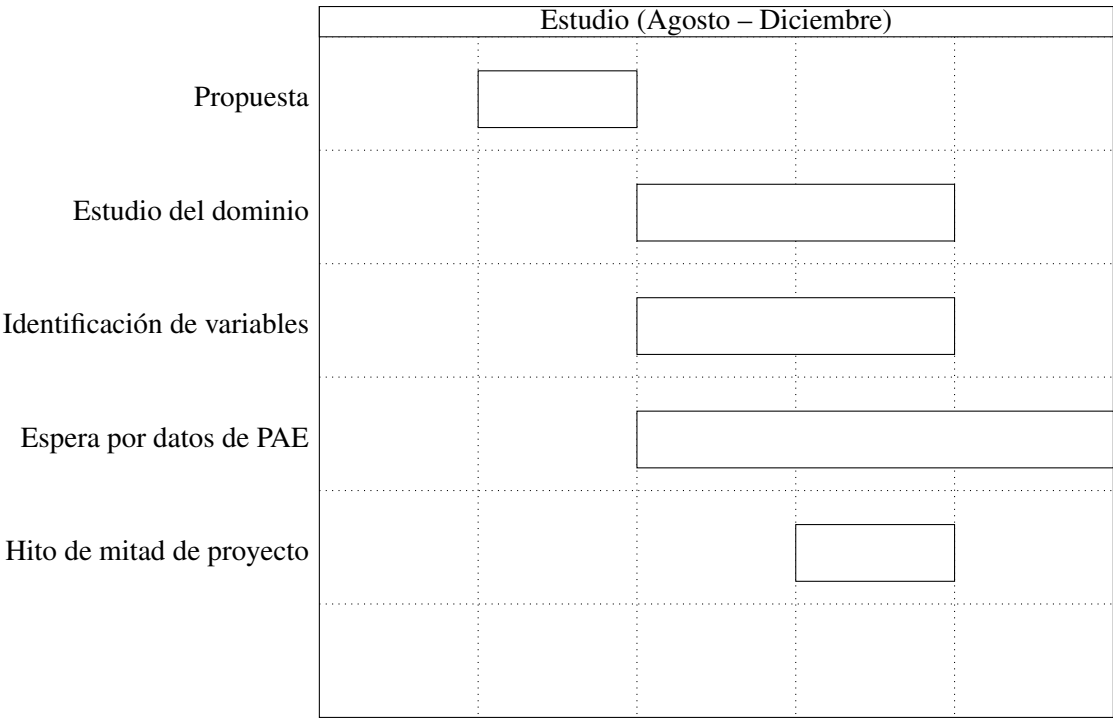


Figura 26: Estudio del dominio

La propuesta fue entregada el 29 de septiembre de 2025 y fue aprobada el 13 de octubre de 2025.
El hito de mitad de proyecto fue entregado el 30 de noviembre de 2025.

14.2 Cronograma del primer cuatrimestre 2026

A continuación, se presenta el cronograma detallado de las actividades desarrolladas durante el primer cuatrimestre de 2026, comprendido entre los meses de marzo y junio.

En este período se llevaron a cabo tareas fundamentales del proyecto, organizadas en tres grandes ejes: la generación y análisis de datos, el entrenamiento y evaluación de modelos, y el desarrollo de la aplicación web.

Cada uno de los diagramas de Gantt incluidos corresponde a una fase específica, en la que se describen las tareas realizadas junto con su duración y distribución temporal a lo largo del cuatrimestre.

La segmentación en tres diagramas permite evidenciar de manera clara el carácter incremental del desarrollo, destacando la existencia de hitos intermedios y una progresión estructurada hacia la finalización del proyecto.

Por último, cabe destacar que la planificación temporal de cada diagrama se encuentra organizada en unidades semanales, lo que facilita una visualización precisa del avance de las actividades.

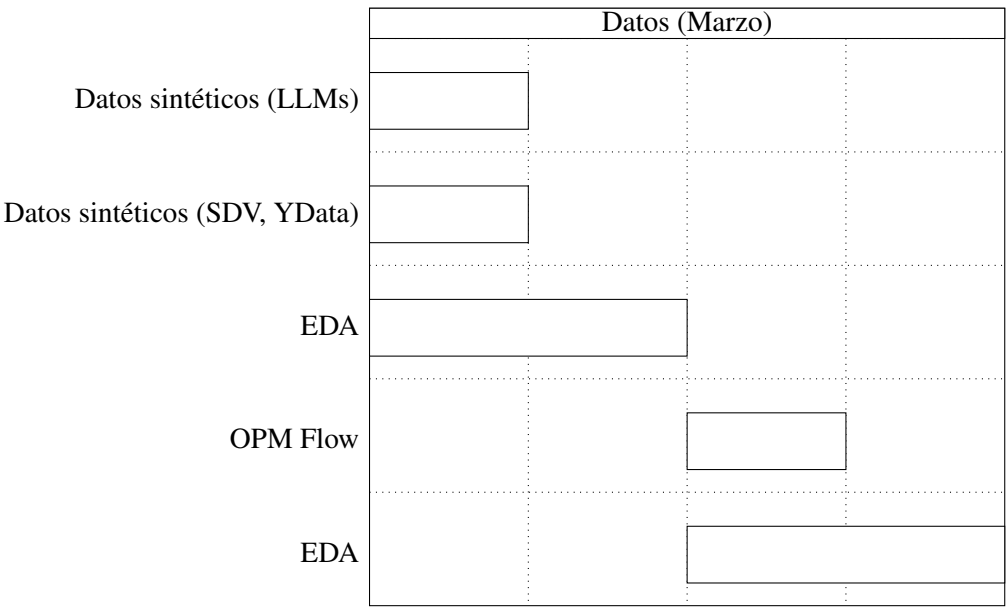


Figura 27: Generación y análisis de datos

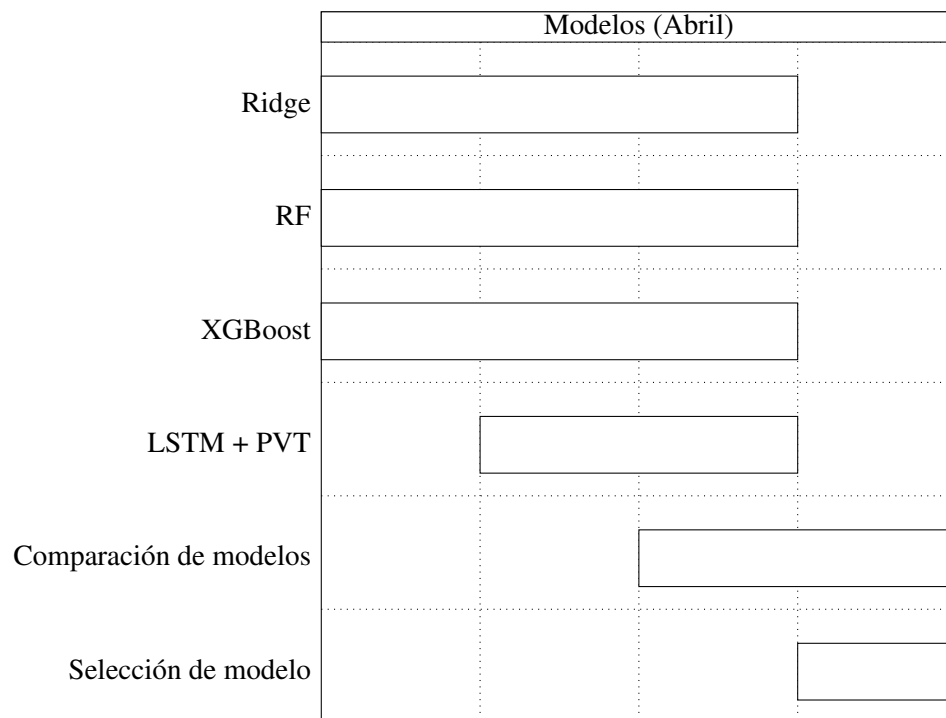


Figura 28: Entrenamiento y evaluación de modelos

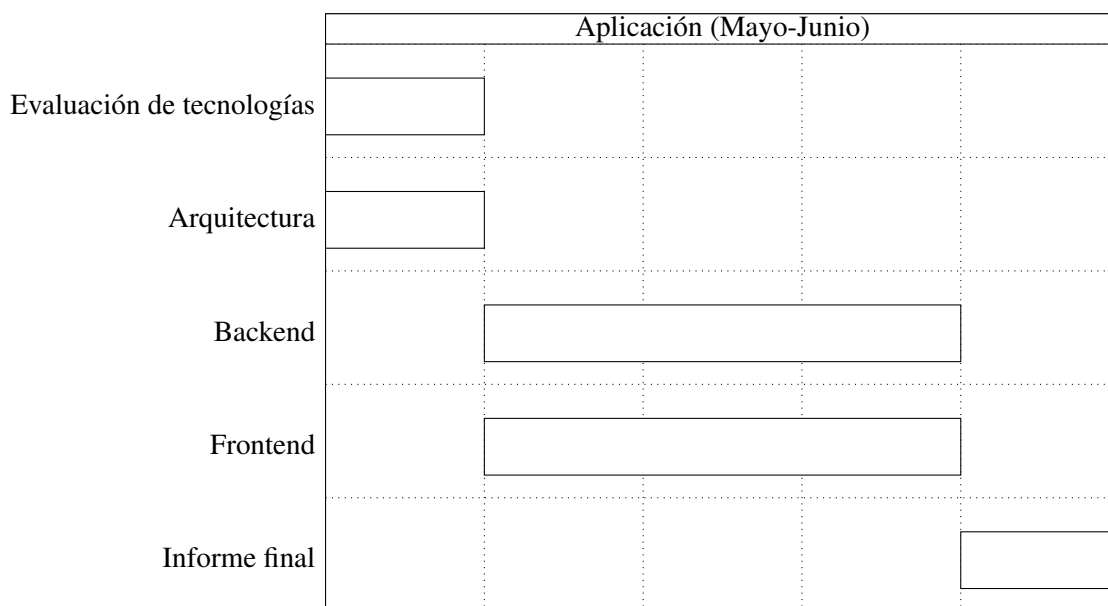


Figura 29: Desarrollo de la aplicación y cierre del proyecto

15 Riesgos materializados y Lecciones aprendidas

15.1 Riesgos materializados

15.1.1. Dependencia de datos de la industria

Uno de los principales riesgos fue la dependencia de datos provenientes de la industria y la escasez de datos reales de presión. Este riesgo se materializó durante el primer cuatrimestre y se mantuvo durante el segundo cuatrimestre.

El equipo esperaba contar con datos provistos por PAE, sin embargo, debido a que el trabajo debía ser de carácter público, no fue posible acceder a dichos datos.

Esta situación implicó un período prolongado de espera e incertidumbre, lo que impactó directamente en la planificación inicial y retrasó el avance del proyecto, particularmente en las etapas de experimentación y modelado.

Se analizaron fuentes de datos públicas, como la información disponible en el Capítulo IV de la Secretaría de Energía de la República Argentina [16]. No obstante, estas fuentes no incluyen datos de presión de reservorios, lo que limitó su utilidad para el problema abordado y reforzó la dependencia de datos privados de la industria.

Como consecuencia, fue necesario redefinir la estrategia de trabajo y optar inicialmente por el uso de datos sintéticos.

Durante el segundo cuatrimestre, se intentó generar estos datos mediante distintas herramientas, pero la falta de confianza en su realismo motivó la adopción de un enfoque alternativo basado en simulaciones físicas utilizando OPM Flow, lo que permitió obtener datasets más representativos.

15.2 Lecciones aprendidas

15.2.1. Comprensión del dominio

En cuanto a las lecciones aprendidas, se destaca en primer lugar la importancia de comprender profundamente el dominio del problema antes de abordar el modelado. El estudio realizado en la primera etapa resultó fundamental para identificar las variables relevantes y orientar correctamente el análisis de datos y la construcción de modelos.

15.2.2. Flexibilidad y adaptabilidad

Asimismo, se evidenció que la planificación de proyectos en contextos reales debe contemplar la incertidumbre asociada al acceso y calidad de los datos. La necesidad de adaptar la estrategia de generación y selección de datos puso de manifiesto la importancia de mantener un enfoque flexible e iterativo.

15.2.3. Importancia de la validación continua

Finalmente, se aprendió que la validación continua con expertos en el dominio es clave para garantizar la calidad y relevancia de los resultados, especialmente en problemas interdisciplinarios donde el conocimiento técnico por sí solo no es suficiente.

16 Impactos sociales y ambientales del proyecto

Conviene aclarar el alcance antes de avanzar: el trabajo es una prueba de concepto, así que lo que sigue son impactos potenciales del enfoque, no resultados ya conseguidos. Estimar bien la presión de un yacimiento importa porque define un margen operativo estrecho, del que dependen la seguridad y la eficiencia de la operación.

16.1 Impacto económico

Una mejor estimación de presión ayuda a planificar la extracción y a elegir el método de recuperación secundaria, lo que baja costos. También hay un ahorro de cómputo: una vez entrenado, el modelo predice en milisegundos, mientras que una simulación numérica completa tarda horas o días. Eso permite explorar más escenarios con menos recursos. Por último, la interfaz web acerca la estimación a ingenieros sin formación en ciencia de datos, que de otro modo dependerían de un especialista.

16.2 Impacto social

Mantener la presión dentro de un margen seguro reduce el riesgo de eventos graves, como un descontrol de pozo o la pérdida de equipos, que pueden costar vidas. Además, el sistema no funciona como una “caja negra”: explica cómo se procesaron los datos y muestra la incertidumbre de la predicción, lo que ayuda a tomar decisiones más informadas.

16.3 Impacto ambiental

Anticipar problemas de presión ayuda a evitar *blowouts* y derrames, con la contaminación que traen. Una mejor gestión de la inyección de agua evita desperdiciar agua y energía. Y reemplazar corridas de simulador por inferencias del modelo baja el consumo energético del modelado.

16.4 Consideración sobre el alcance

Hay que reconocer que la herramienta opera dentro de la industria de los hidrocarburos. Su aporte no es extraer más, sino hacer más seguras y eficientes operaciones que ya existen. La veta ambiental positiva está ahí: en la prevención de daños y en el uso más cuidadoso del agua y la energía.

17 Trabajos futuros

El trabajo deja varias líneas abiertas, tanto del lado del modelo como de la aplicación.

17.1 Física embebida (PINN de balance de materiales)

La experimentación mostró que ningún modelo generaliza bien a un campo cuyo régimen no vio durante el entrenamiento, como pasó con SPE9, que cruza el punto de burbuja y libera gas. Una salida es dejar de aprender todo desde los datos y meter la física adentro del modelo. La ecuación de balance de materiales describe cómo cae la presión a medida que se produce e inyecta fluido; usada como esqueleto del modelo, es ella la que carga con la extrapolación a un campo nuevo.

Las exploraciones preliminares en esta dirección son alentadoras: una versión basada en un modelo de tanque, con un solo parámetro físico ajustable, generaliza a un campo no visto donde los modelos puramente data-driven daban R^2 negativo. Sumar el término de gas libre (B_g) ayuda además en el régimen que cruza el punto de burbuja. El paso siguiente es desarrollar esto como un PINN completo, que combine la ecuación física con una corrección aprendida, y evaluarlo bajo el mismo protocolo *leave-one-field-out*. Esta línea también concreta el uso de PINNs que se anticipa en el estado del arte.

17.2 Calibración few-shot

Los experimentos de cobertura dejaron una observación clara: la predicción *zero-shot* a un campo nuevo falla por el cambio de dominio, pero alcanza con muy pocos datos del campo nuevo para corregirlo. Agregar una sola simulación del campo objetivo al entrenamiento lleva el R^2 de alrededor de 0,14 a 0,89.

En lugar de perseguir una generalización *zero-shot* perfecta, conviene diseñar un flujo de calibración: ante un yacimiento nuevo, conseguir un puñado de simulaciones (o algo de datos reales) y reajustar el modelo. Queda por cuantificar cuántas corridas hacen falta para alcanzar una precisión objetivo. La implicancia práctica es que baja la barrera de datos para usar la herramienta en un campo que no estaba representado.

17.3 Más campos y datos reales

El trabajo se apoyó en tres campos sintéticos: Volve, Norne y SPE9. Ampliar ese conjunto, sobre todo con datos reales de producción y presión de la industria, permitiría un *leave-one-field-out* más exigente y, sobre todo, comprobar si las conclusiones obtenidas con datos sintéticos se sostienen frente a datos de campo.

17.4 Validación contra métodos físicos (PTA/EBM)

La propuesta inicial planteaba contrastar las predicciones del modelo con las técnicas clásicas de la ingeniería de reservorios: el Análisis de Transientes de Presión (PTA) y la Ecuación de Balance de Materiales (EBM). Esa comparación quedó pendiente. Hacerla ubicaría al enfoque de machine learning respecto de las herramientas ya establecidas y serviría para chequear la consistencia física de las estimaciones.

17.5 Mejoras en la aplicación web

El modelo que sirve hoy la aplicación usa un encoder que comprime la tabla PVT junto con features de superficie. Una mejora natural es migrar esas features a balance de materiales (voidage neto), que son más físicas y transfieren mejor entre campos. También conviene mostrar la incertidumbre de la predicción como media y desvío sobre varias semillas, en lugar de una corrida única, y avanzar hacia un despliegue, ya que por ahora la aplicación corre solo en entorno local.

18 Conclusiones

El objetivo planteado en la propuesta era construir un sistema basado en machine learning para estimar la presión de un reservorio, pensado para ingenieros sin formación en ciencia de datos. Ese objetivo se cumplió. El trabajo produjo el pipeline completo, desde la generación de datos sintéticos con OPM Flow (40 simulaciones de Volve y Norne, más SPE9 para la validación multi-campo), pasando por las *features* adimensionales basadas en el balance de materiales y la comparación de cuatro familias de modelos, hasta la aplicación web que sirve las predicciones y explica cómo se procesaron los datos. Todos los componentes funcionan integrados.

La conclusión técnica más importante no estaba en la propuesta original, sino que surgió durante la experimentación. Predecir la presión del campo con el que el modelo fue entrenado es un problema prácticamente resuelto: las cuatro familias evaluadas superan un R^2 de 0.9 *in-distribution*. El desafío real es generalizar entre reservorios. El mismo modelo que alcanza un R^2 de 0.99 sobre Norne puede dar predicciones físicamente absurdas sobre Volve. Este *domain shift* es lo que define la dificultad de aplicar machine learning en este dominio.

Frente a ese problema, encontramos que los factores que determinan la transferencia entre campos son metodológicos antes que algorítmicos. Anclar la predicción a la presión inicial del reservorio y normalizar las cantidades por el volumen poral importó más que la elección entre Ridge, XGBoost o LSTM. Con el target anclado, hasta los modelos de árboles transfieren a un campo no visto; sin el anclaje, ninguna familia lo logra. A igual representación, ningún modelo domina en todos los campos. El LSTM se destacó por su robustez pareja entre campos, y por eso es el que sirve la aplicación web.

El experimento *leave-one-field-out* marcó además el límite del enfoque puramente *data-driven*: ningún modelo extrapola a un régimen termodinámico que no vio durante el entrenamiento, como pasa con SPE9, cuya presión cruza el punto de burbuja y libera gas. La física que no está en los datos de entrenamiento no se puede interpolar. Hay dos salidas: incorporarla al modelo, que es la línea de física embebida y PINNs que se desarrolla en Trabajos Futuros, o compensarla con datos del campo objetivo. Sobre esta segunda, los experimentos de cobertura indican que con muy pocas simulaciones del campo nuevo alcanza para recuperar el desempeño mediante calibración. El modo de uso realista del sistema, entonces, no es la generalización perfecta sin datos, sino calibrar con poca información del campo objetivo.

En cuanto a la gestión, el riesgo más temido del anteproyecto, la dependencia de datos de la industria, se materializó por completo. La viabilidad del proyecto dependió de la capacidad de adaptación: el paso de esperar datos reales a generarlos con un simulador físico terminó marcando el rumbo del trabajo, y las reuniones quincenales con los directores permitieron detectar a tiempo los desvíos de alcance y de modelado. La lección que nos llevamos es que en un proyecto de datos aplicado a un dominio industrial, la estrategia de datos merece el mismo nivel de planificación que el modelado.

Por último, corresponde delimitar el alcance de estas conclusiones. Todos los resultados se obtuvieron sobre datos simulados. La transferencia a datos de campo reales no fue demostrada y es la validación pendiente más importante, junto con el contraste contra los métodos físicos tradicionales (PTA y EBM), que estaba comprometido en la propuesta y quedó fuera de esta etapa. Dentro de esos límites, el trabajo deja un sistema funcional, una idea clara de dónde aporta el machine learning y dónde necesita de la física, y una dirección de continuación concreta.

19 Aportes individuales

En la siguiente tabla se muestran los aportes individuales de cada integrante:

	Franco Bragantini	Máximo Palopoli	Mateo Rico	Alan Valdevenito
Cantidad de horas insumidas	-	385	385	-
Elaboración de la propuesta	-	20	22	-
Estudio del dominio	-	48	45	-
Elaboración del hito de mitad de proyecto	-	14	15	-
Investigación sobre generación de datos sintéticos	-	25	27	-
Generación de datos sintéticos	-	30	81	-
Análisis exploratorio de datos	-	40	38	-
Entrenamiento de modelo	-	110	109	-
Desarrollo de la aplicación web	-	73	20	-
Elaboración del informe final	-	25	28	-

20 Anexos

A continuación se listan los repositorios en los cuales se aloja el código del proyecto:

- Aplicación web (frontend y backend) <https://github.com/tpp-grupo-166/reservoir-pressure-web>
- Datasets y modelos de machine learning <https://github.com/tpp-grupo-166/opm-datasets>

En este repositorio también están incluidos tanto los modelos de reservorio como la herramienta que utilizamos para generar los datos sintéticos.

21 Referencias

- [1] Barriol, Y. (2005). *Las presiones de las operaciones de perforación y producción*. Oil-field Review, 22. <https://usuarios.geofisica.unam.mx/gvazquez/geoquimpetrolFI/zonadesplegar/Lecturas/Las%20presiones%20de%20las%20operaciones%20%20de%20perforacion%20y%20producc.pdf>
- [2] PetroWiki (SPE). *Pressure transient testing*. https://petrowiki.spe.org/Pressure_transient_testing
- [3] PetroWiki (SPE). *Material balance in oil reservoirs*. https://petrowiki.spe.org/Material_balance_in_oil_reservoirs
- [4] Ali, A. (2021). *Machine learning based data-driven methods for modelling and simulation of pressure dynamics and fluid flow in natural gas reservoirs* (Tesis doctoral, University of Sheffield). https://etheses.whiterose.ac.uk/id/eprint/29161/1/Aliyuda_Final_Thesis.pdf
- [5] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). *Physics-Informed Neural Networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*. Journal of Computational Physics, 378, 686-707. <https://www.sciencedirect.com/science/article/abs/pii/S0021999118307125>
- [6] Maarouf, A., Tahir, S., Su, S., Kloucha, C. K., & Mustapha, H. (2023). *A comparative study for deep-learning-based methods for automated reservoir simulation*. SPE Reservoir Characterisation and Simulation Conference and Exhibition. https://www.researchgate.net/publication/367379296_A_Comparative_Study_for_Deep-Learning-Based-Methods_for_Automated_Reservoir_Simulation
- [7] Chen, X., Zhang, K., Ji, Z., Shen, X., Liu, P., Zhang, L., Wang, J., & Yao, J. (2023). *Progress and Challenges of Integrated Machine Learning and Traditional Numerical Algorithms: Taking Reservoir Numerical Simulation as an Example*. Mathematics, 11(21), 4418. <https://www.mdpi.com/2227-7390/11/21/4418>
- [8] El Hannaoui, M. (2025). *Artificial Intelligence in Oil and Gas: How AI is Transforming Reservoir Engineering*. Novi Labs. <https://novilabs.com/blog/ai-in-reservoir-engineering-how-artificial-intelligence-is-transforming-oil-gas/>
- [9] Open Porous Media (OPM) Initiative. *OPM Flow — simulador de reservorios open source*. <https://opm-project.org/>

- [10] SLB (Schlumberger). *ECLIPSE — formato y simulador de referencia de reservorios*. <https://www.software.slb.com/products/eclipse>
- [11] Open Porous Media (OPM). *Norne benchmark case* (repositorio opm-tests). <https://github.com/OPM/opm-tests/tree/master/norne>
- [12] Equinor. *Volve field data set* (datos abiertos del campo Volve). <https://www.equinor.com/energy/volve-data-sharing>
- [13] PetroWiki (SPE). *Voidage replacement ratio*. https://petrowiki.spe.org/Voidage_replacement_ratio
- [14] Equinor. *resdata — librería para leer los archivos de salida del simulador ECLIPSE / OPM Flow*. <https://github.com/equinor/resdata>
- [15] Society of Petroleum Engineers. *SPE Comparative Solution Project* (modelo benchmark SPE9). <https://www.spe.org/web/csp/>
- [16] Secretaría de Energía, República Argentina. *Portal de Datos Abiertos de Energía*. <http://datos.energia.gob.ar/>